



VERIFIZIERTE C-LIBRARY FÜR LOCKFREIE DATENSTRUKTUREN

ENTWICKLUNG DER LOCKFREIEN DATENSTRUKTUREN

Software-Entwicklungspraktikum (SEP)

Sommersemester 2023

Technischer Entwurf

Auftraggeber

Technische Universität Braunschweig

Institute of Theoretical Computer Science

Prof. Dr. rer. nat. Roland Meyer

IZ 346, i.e. Informatikzentrum, 3rd floor, office 346, Mühlenpfordtstr. 23

D-38106 Braunschweig

Betreuer: Sören van der Wall

Auftragnehmer:

Name	E-Mail-Adresse
Keanu Wirz	k.wirz@tu-braunschweig.de
Nazli Cesur	n.cesur@tu-braunschweig.de
Tristen Vaeckenstedt	t.vaeckenstedt@tu-braunschweig.de
Jacob Müller	jacob.mueller@tu-braunschweig.de
Mohamed Amine Halila	m.halila@tu-braunschweig.de
Omar Zitouni	o.zitouni@tu-braunschweig.de

Braunschweig, 28. Juni 2023

Bearbeiterübersicht

Kapitel	Autoren
1	Nazli
1.1	Nazli
2	Mohamed Amine und Omar
2.1	Mohamed Amine und Omar
2.2	Mohamed Amine und Omar
2.3	Mohamed Amine und Omar
2.4	Mohamed Amine und Omar
2.5	Mohamed Amine und Omar
2.6	Mohamed Amine und Omar
2.7	Mohamed Amine und Omar
2.8	Mohamed Amine und Omar
2.9	Mohamed Amine und Omar
2.10	Mohamed Amine und Omar
3	Keanu
3.1	Keanu
3.2	Keanu
3.3	Keanu
4	Jacob
5	Keanu
5.1	Keanu
5.2	Keanu
5.3	Keanu
5.4	Keanu
5.5	Keanu
5.6	Keanu
6	Tristen
7	Omar
8	Omar, Mohamed Amine und Nazli
9	Tristen
9.1	Tristen
9.2	Tristen
9.3	Tristen
10	Alle

Inhaltsverzeichnis

1	Einleitung	6
1.1	Projektdetails	9
1.1.1	Compare-and-Swap Funktion	9
1.1.2	Benchmark und Visualisierung	10
1.1.3	Freigabe des Nodes	10
2	Analyse der Produktfunktionen	11
2.1	Analyse von Funktionalität <F10>: <Push im Stack>	12
2.2	Analyse von Funktionalität <F20>: <Pop aus Stack>	13
2.3	Analyse von Funktionalität <F30>: <Enqueue>	14
2.4	Analyse von Funktionalität <F40>: <Dequeue>	15
2.5	Analyse von Funktionalität <F50>: <Insert>	16
2.6	Analyse von Funktionalität <F60>: <Delete>	17
2.7	Analyse von Funktionalität <F70>: <Find>	18
2.8	Analyse von Funktionalität <F80>: <Search>	19
2.9	Analyse von Funktionalität <F90>: <Visualisieren>	20
2.10	Analyse von Funktionalität <F100>: <Benchmark>	21
3	Resultierende Softwarearchitektur	22
3.1	Komponenten-Spezifikation	22
3.2	Schnittstellenspezifikation	23
3.3	Protokolle für die Benutzung der Komponenten	26
4	Verteilungsentwurf	27
5	Implementierungsentwurf	28
5.1	Implementierung von Komponente <10>: <Set>:	28
5.1.1	Paket-/Klassendiagramm	28
5.1.2	Erläuterung	29
5.2	Implementierung von Komponente <11>: <Queue>:	30
5.2.1	Paket-/Klassendiagramm	30
5.2.2	Erläuterung	30

5.3	Implementierung von Komponente <12>: <Stack>:	31
5.3.1	Paket-/Klassendiagramm	31
5.3.2	Erläuterung	31
5.4	Implementierung von Komponente <20>: <Benchmark>:	32
5.4.1	Paket-/Klassendiagramm	32
5.4.2	Erläuterung	33
5.5	Implementierung von Komponente <30>: <Inputwindow>:	34
5.5.1	Paket-/Klassendiagramm	34
5.5.2	Erläuterung	34
5.6	Implementierung von Komponente <31>: <Graph>:	36
5.6.1	Paket-/Klassendiagramm	36
5.6.2	Erläuterung	36
6	Datenmodell	38
7	Konfiguration	39
8	Änderungen gegenüber Fachentwurf	40
9	Erfüllung der Kriterien	41
9.1	Muss-Kriterien	41
9.2	Soll-Kriterien	42
9.3	Kann-Kriterien	42
9.4	Abgrenzungskriterien	43
10	Glossar	44

Abbildungsverzeichnis

1.1	Gesamter Anwendungsprozess als Aktivitätsdiagramm	8
1.2	CAS Aktivitätsdiagramm	9
1.3	Allokation der Nodes als Zustandsdiagramm	10
2.1	Pushen im Stack	12
2.2	Pop aus Stack	13
2.3	Enqueue	14
2.4	Dequeue	15
2.5	Insert im Set	16
2.6	Delete	17
2.7	Find	18
2.8	Search in Set	19
2.9	Visualisieren	20
2.10	Benchmark	21
3.1	Komponentendiagramm für Library und UI	22
3.2	Statechart für die korrekte Verwendung der Datenstrukturen	26
4.1	Verteilungsdiagramm	27
5.1	Klassendiagramm Set $\langle C10 \rangle$	28
5.2	Klassendiagramm Queue $\langle C11 \rangle$	30
5.3	Klassendiagramm Stack $\langle C12 \rangle$	31
5.4	Klassendiagramm Benchmark $\langle C20 \rangle$	32
5.5	Klassendiagramm Inputwindow $\langle C30 \rangle$	34
5.6	Klassendiagramm Graph $\langle C31 \rangle$	36

1 Einleitung

Dieses Dokument dient als technischer Entwurf für das Projekt lockfreie Datenstrukturen, welches dem Auftraggeber ein Überblick über die Funktionen des Produkts verschaffen soll. Als Erstes wird ein Aktivitätsdiagramm angegeben, welches über das Softwaresystem eine Übersicht gibt. Im Kapitel 1 werden die komplizierten und wichtigen Prozesse von unserer Software aufgelistet und diese detailliert erklärt. In Kapitel 2 wird das Verhalten von Produktfunktionen mit Sequenzdiagrammen dargestellt und erklärt. Daraufhin wird weiter auf die Softwararchitektur sowie die Kommunikation der Komponenten eingegangen. Weitergehend wird auch ein Verteilungsdiagramm gegeben, welches darstellt wie unsere Software verteilt ist. Unter anderem geben wir eine genauere Einsicht auf die Implementierung unserer Komponenten mithilfe von Klassendiagrammen. Im Schluss behandeln wir die geplante Konfiguration der Software, beleuchten Änderungen die stattgefunden haben, sowie die Erfüllung unserer Kriterien.

Das Ziel dieses Projekts ist es, eine lockfreie Library für die Programmiersprache C zu entwickeln. Die Library bietet eine effiziente und fehlerfreie Möglichkeit, Synchronisationsmechanismen in Mehrkernsystemen zu implementieren, ohne auf Sperren (Locks) zurückgreifen zu müssen. Durch den Verzicht auf Locks wird das Problem von Wettlaufsituationen (Race Conditions) und Deadlocks eliminiert, während gleichzeitig die Leistungsfähigkeit des Systems gesteigert wird. Wie in der Abbildung 1.1 beschrieben, ermöglicht unsere Visualisierung dem Nutzer, die Anzahl der Threads, sowie die Anzahl der Operationen anzugeben. Die Library bietet eine Reihe von Datenstrukturen wie z.B. Queue, Stack und List, die auf den Prinzipien der Thread-Sicherheit und atomaren Operationen basieren. Die Datenstrukturen haben jeweils die Einfüge-Operationen(push, enqueue, insert) und Löschoperationen(pop, dequeue, delete).

Beim Einfügen wird erst ein neuer Node alloziert. Beim Inserten von Set, anders als Stack und Queue, wird erst die richtige Stelle gesucht und gecheckt, ob das Element, das den insert ausführt, schon im Set existiert. Wenn das Element schon existiert, braucht man keinen insert ausführen. Die Funktion ruft return auf. Wenn nicht, wird die aktuelle Datenstruktur, wie bei Queue und Stack gelesen. Danach wird der Node lokal initialisiert, wie es zur Datenstruktur passt. Es wird im Schluss mit der Compare and Swap Funktion (CAS) gecheckt, ob der gelesene Wert mit der aktuellen Datenstruktur immer noch übereinstimmen. Wenn sie übereinstimmen, heißt das, dass kein andere Thread die Operation unterbrochen hat. Der Thread darf das Element hinzufügen bzw. die Datenstruktur aktualisiert sich als lokale Datenstruktur. Wenn sie nicht

übereinstimmen, heißt das, dass ein anderer Thread die Operation unterbrochen hat. Der Thread muss nochmal die Datenstruktur lesen und mit CAS checken.

Beim Löschen von Set, anders als Stack und Queue, wird erst das löschende Element gesucht, ob es schon im Set existiert. Wenn es nicht gefunden wird, terminiert die Funktion. Wenn es gefunden wird, wird der aktuelle Stack gelesen und die Datenstruktur wird lokal aktualisiert, sodass das Element entfernt wird. Es wird am Schluss mit der Compare and Swap Funktion gecheckt, ob der gelesene Wert mit der aktuellen Datenstruktur immer noch übereinstimmt. Wenn sie übereinstimmen, heißt das, dass kein andere Thread die Operation unterbrochen hat. Der Thread darf das Element hinzufügen bzw. die Datenstruktur aktualisiert sich als lokale Datenstruktur. Wenn sie nicht übereinstimmen, heißt das, dass ein anderer Thread die Operation unterbrochen hat. Der Thread muss nochmal die Datenstruktur lesen und mit CAS checken.

Das Userinterface bietet auch die Möglichkeit, die Ergebnisse grafisch darzustellen. Nach Abschluss der Operationen generiert das Interface ein Geschwindigkeitsdiagramm, das die gemessene Leistung der lockfreien Datenstrukturlibrary in Bezug auf die angegebenen Parameter visualisiert. Dadurch kann der Benutzer die Effizienz und Skalierbarkeit der Library direkt beobachten und analysieren.

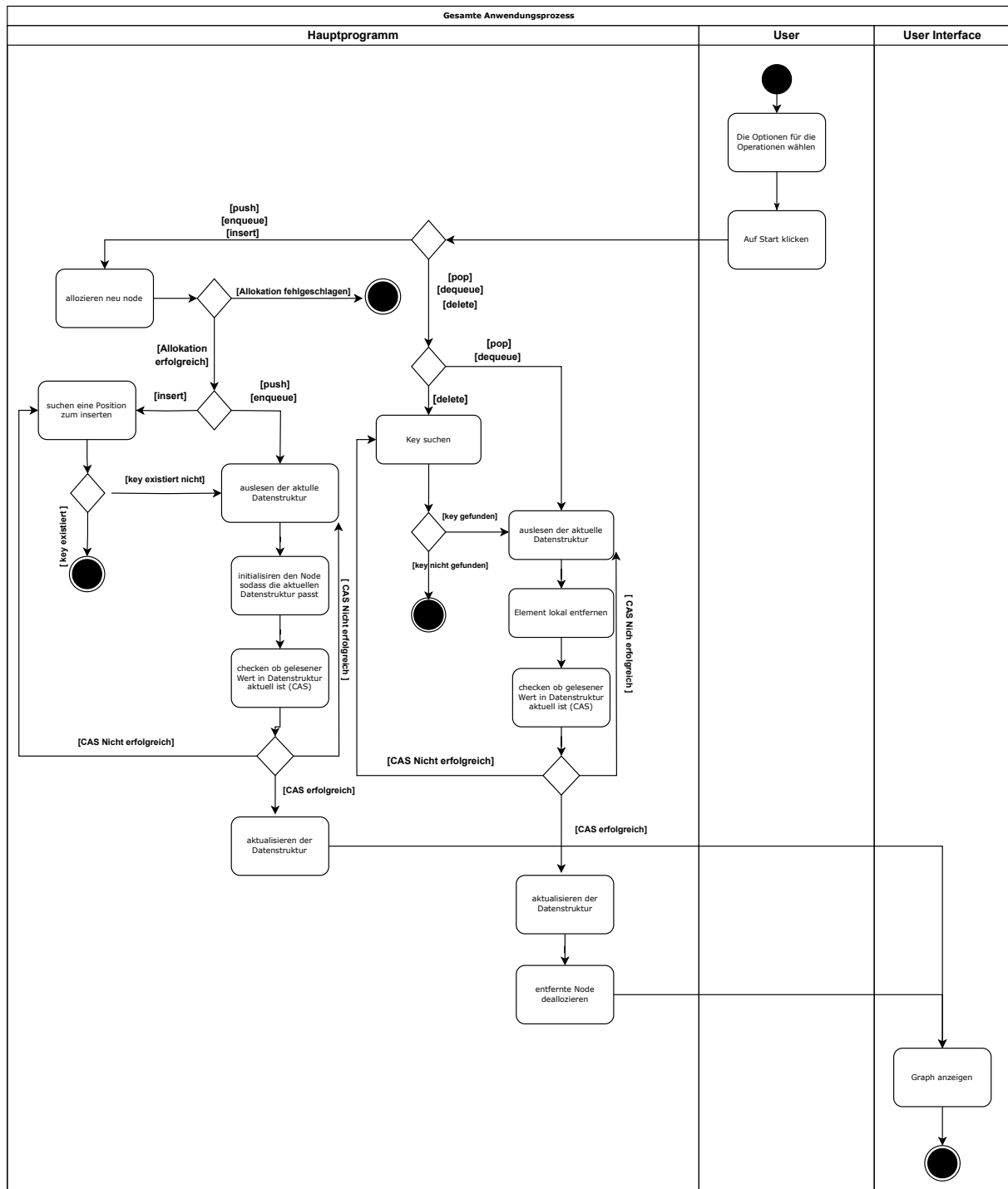


Abbildung 1.1: Gesamter Anwendungsprozess als Aktivitätsdiagramm

1.1 Projektdetails

In diesem Abschnitt werden komplizierte Sachverhalten detailliert erklärt, die in der Abbildung 1.1 nicht vertieft werden.

- Compare and Swap Funktion
- Benchmark und Visualisierung
- Speicheradresse Verwaltung

1.1.1 Compare-and-Swap Funktion

Diese Funktion wird benutzt, um Daten atomar zu aktualisieren. Die grundlegende Funktionsweise von Compare-and-Swap besteht darin, den aktuellen Wert einer Speicherzelle mit einem erwarteten Wert zu vergleichen. Wenn der aktuelle Wert mit dem erwarteten Wert übereinstimmt, wird ein neuer Wert in die Speicherzelle geschrieben. Andernfalls wird keine Aktualisierung durchgeführt. Im Zusammenhang von unserem Projekt ist der CAS wichtig um in einer einzigen Operation Veränderungen vorzunehmen, die nicht unterbrochen werden kann.

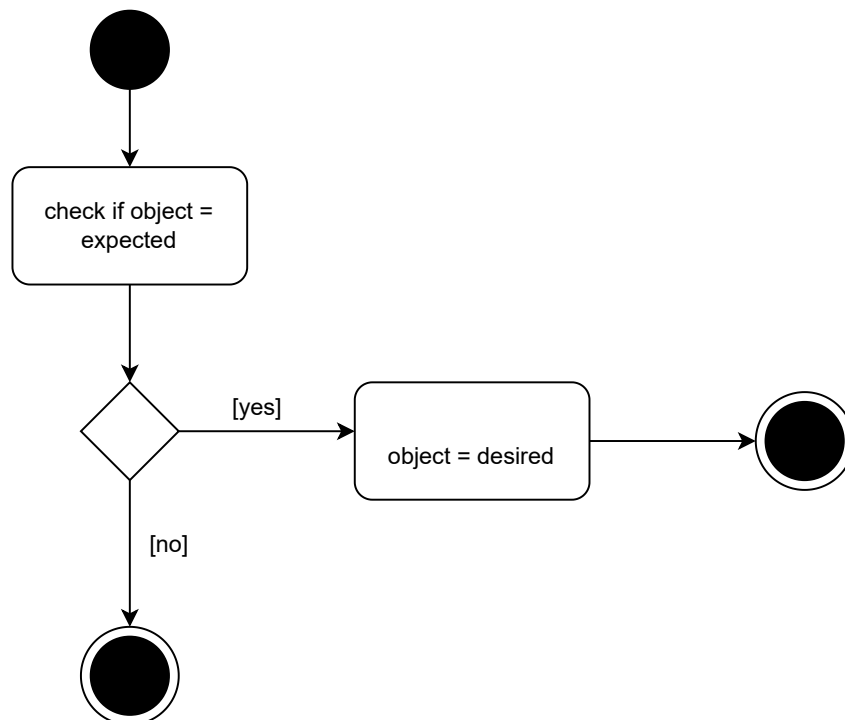


Abbildung 1.2: CAS Aktivitätsdiagramm

1.1.2 Benchmark und Visualisierung

Um die Effizienz und Leistungsfähigkeit der lockfreien Datenstrukturlibrary zu validieren, werden Benchmarks durchgeführt. Diese Benchmarks messen und vergleichen die Leistung der lockfreien Datenstrukturen mit herkömmlichen Lock-basierten Ansätzen. Die Ergebnisse werden in einem Benchmark-Diagramm präsentiert, das die Vorteile und Verbesserungen der lockfreien Implementierung verdeutlicht. Der Benchmark erstellt die gewünschte Anzahl der Threads, die dann die gewünschte Anzahl der Operationen ausführt und dann die Ergebnisse der Latenz und Overalltime Bestimmung in eine .csv Datei speichert, die dann im Visualisierungsprogramm ausgewertet werden. Für jeden Operationsaufruf wird Clock-Timer gestartet. Nachdem Aufruf fertig durchgeführt ist, wird Timer gestoppt. Damit berechnen wir die durchschnittliche Dauer von Datenstruktur Funktionen. Die Dauern wird temporär gespeichert und diese Werte in Graph angezeigt.

1.1.3 Freigabe des Nodes

Es kommt zu ungewünschtem Verhalten, wenn auf eine Speicheradresse zugegriffen wird, nachdem sie bereits freigegeben wird. Um das zu verhindern, wird ein Reference Counter benutzt. Vor dem Freigeben einer Adresse wird erst gecheckt, ob gleiche Adresse von anderen Prozessen benutzt wird. Diese Überprüfung geschieht durch eine counter value innerhalb einer Node. Bei jedem Zugriff inkrementiert der counter und dekrementiert, wenn die Adresse nicht mehr gebraucht ist. Wenn Counter auf 0 steht, kann der belegte Speicher freigegeben werden.

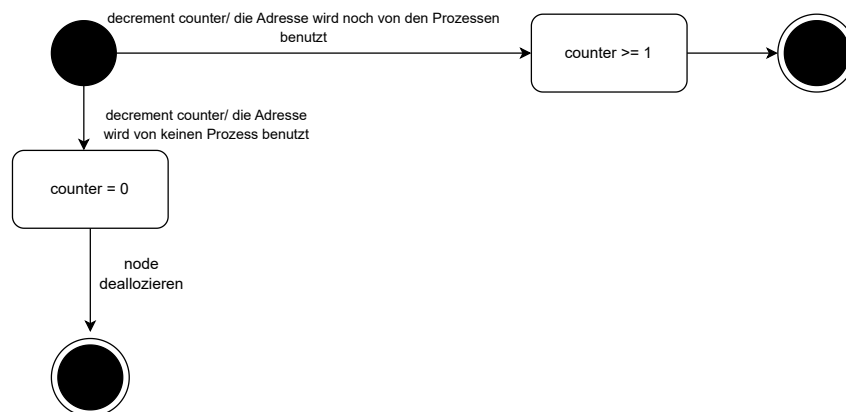


Abbildung 1.3: Allokation der Nodes als Zustandsdiagramm

2 Analyse der Produktfunktionen

Im folgenden Kapitel werden die im Pflichtenheft definierten Produktfunktionen anhand ihres Verhaltens analysiert. Dies wird mit Sequenzdiagrammen veranschaulicht, um die Kommunikation und die Interaktion von Objekten grafisch darzustellen.

2.1 Analyse von Funktionalität <F10>: <Push im Stack>

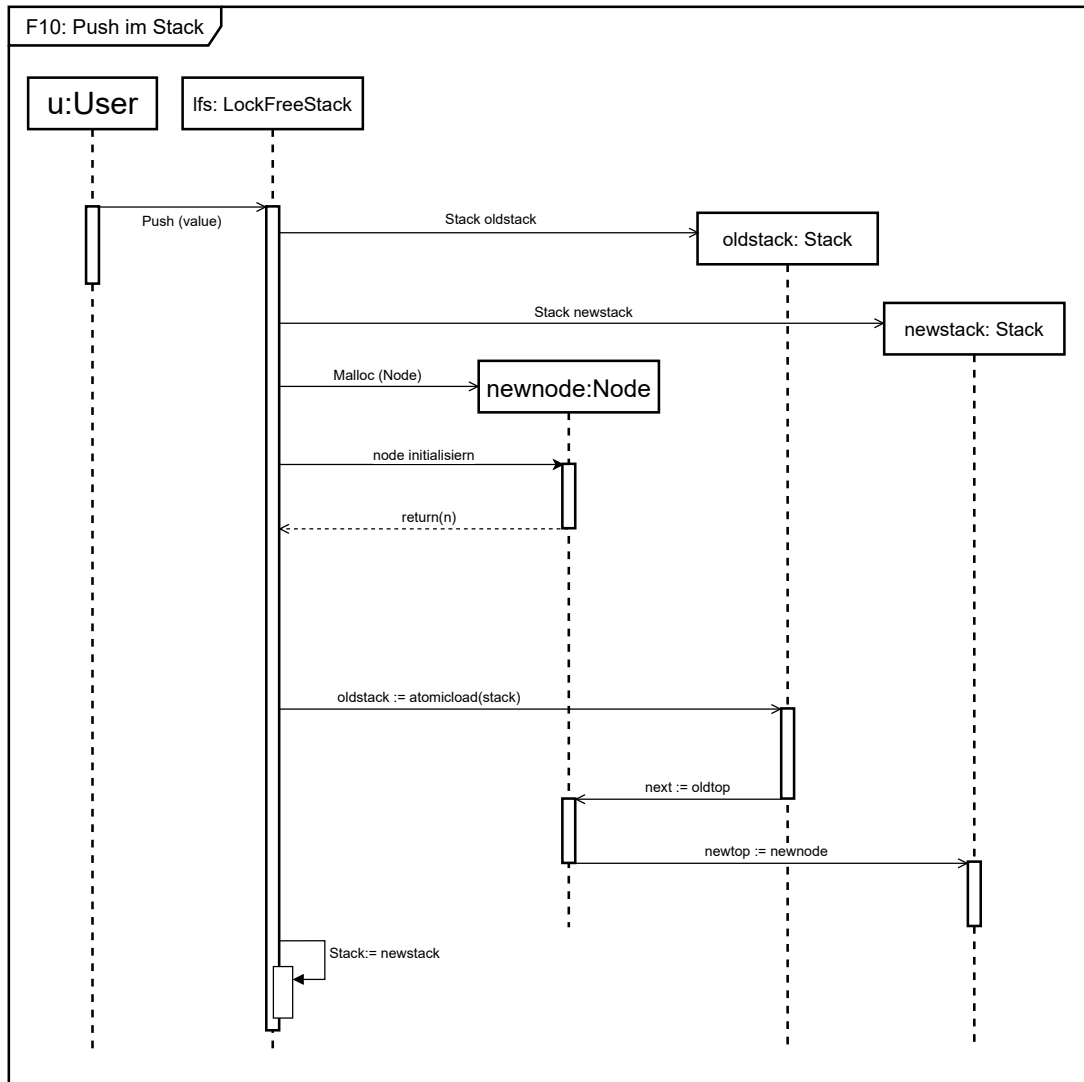


Abbildung 2.1: Pushen im Stack

<F10>: In diesem Sequenzdiagramm, wird der Prozess Push im Stack dargestellt. Dies geschieht, indem der User die funktion `push(value)` aufruft. Hinterher werden lokale Objekte `newstack`, `oldstack` erstellt, um den alten und den neuen Stack zu halten. Danach wird ein neuer Knoten `newnode` im Speicher reserviert und der Wert `value` im `data`-feld der Knoten zugewiesen. Nachfolgend wird der globale Stack im `oldstack` durch atomares Laden gelesen und der `next` zeiger von der neuen Node wird auf den top von `oldstack` gesetzt. Anschließend wird `newstack` aktualisiert, indem sein top auf der neue Node `newnode` zeigt. Endlich wird der globalen Stack durch den neuen Stack `newstack` ersetzt.

2.2 Analyse von Funktionalität <F20>: <Pop aus Stack>

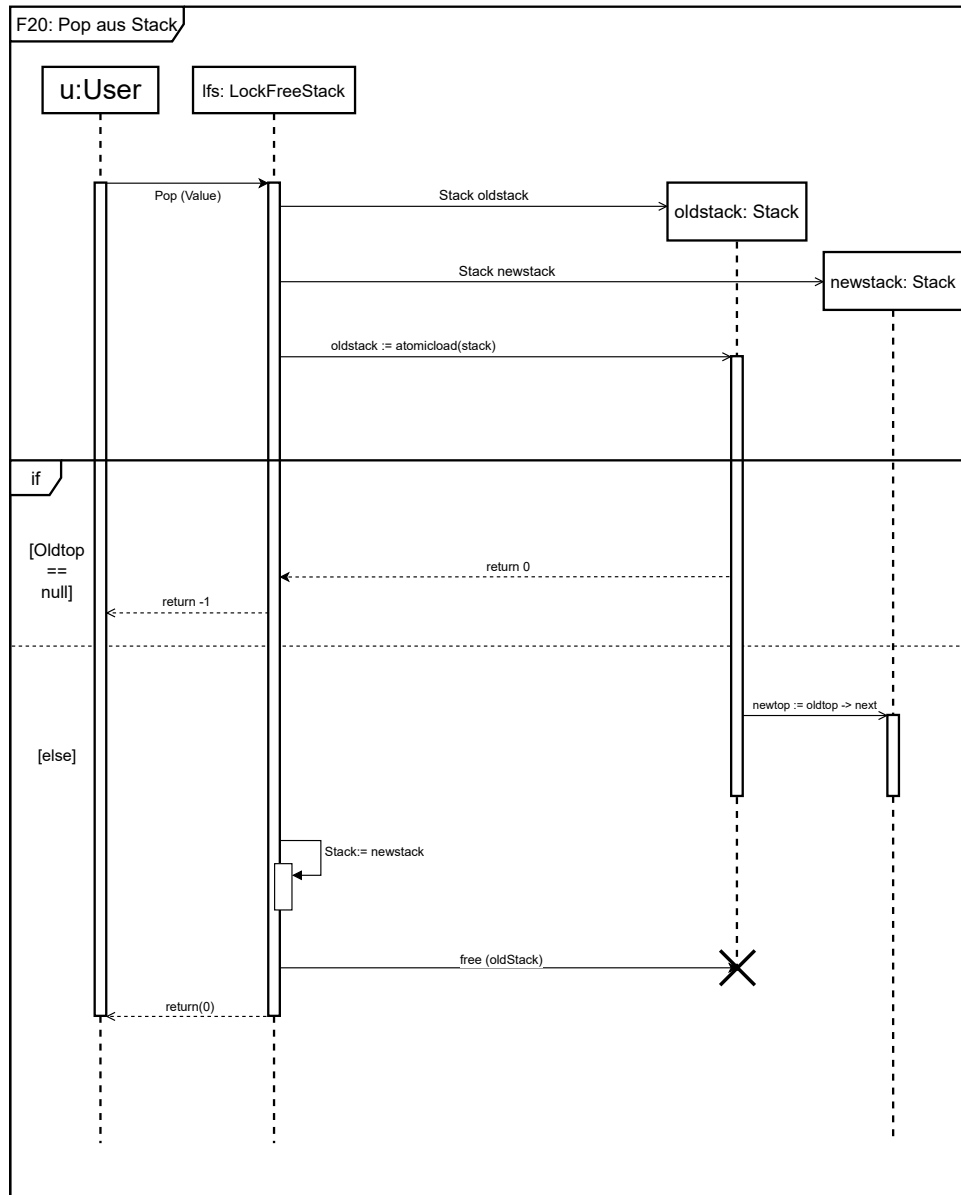


Abbildung 2.2: Pop aus Stack

<F20>: In diesem Sequenzdiagramm, wird der Prozess des Pop aus Stack dargestellt. Dies geschieht in dem der User die funktion `pop(Stack)` aufruft. Hinterher werden lokale Objekte `newstack`, `oldstack` erstellt, um den alten und den neuen Stack zu halten. Dann wird geprüft, ob die `top Node` von `oldstack` gleich `NULL` ist oder nicht. Falls ja, dann ist der Stack leer und die Funktion gibt -1 zurück. Ansonsten wird "newstack" aktualisiert, indem sein `top` auf den `next` Knoten von "top" des `oldstack` zeigt. Endlich wird der globalen Stack durch ein neuer Stack ("`newstack`") ersetzt und der alte `top` Knoten des Stack wird freigegeben.

2.3 Analyse von Funktionalität <F30>: <Enqueue>

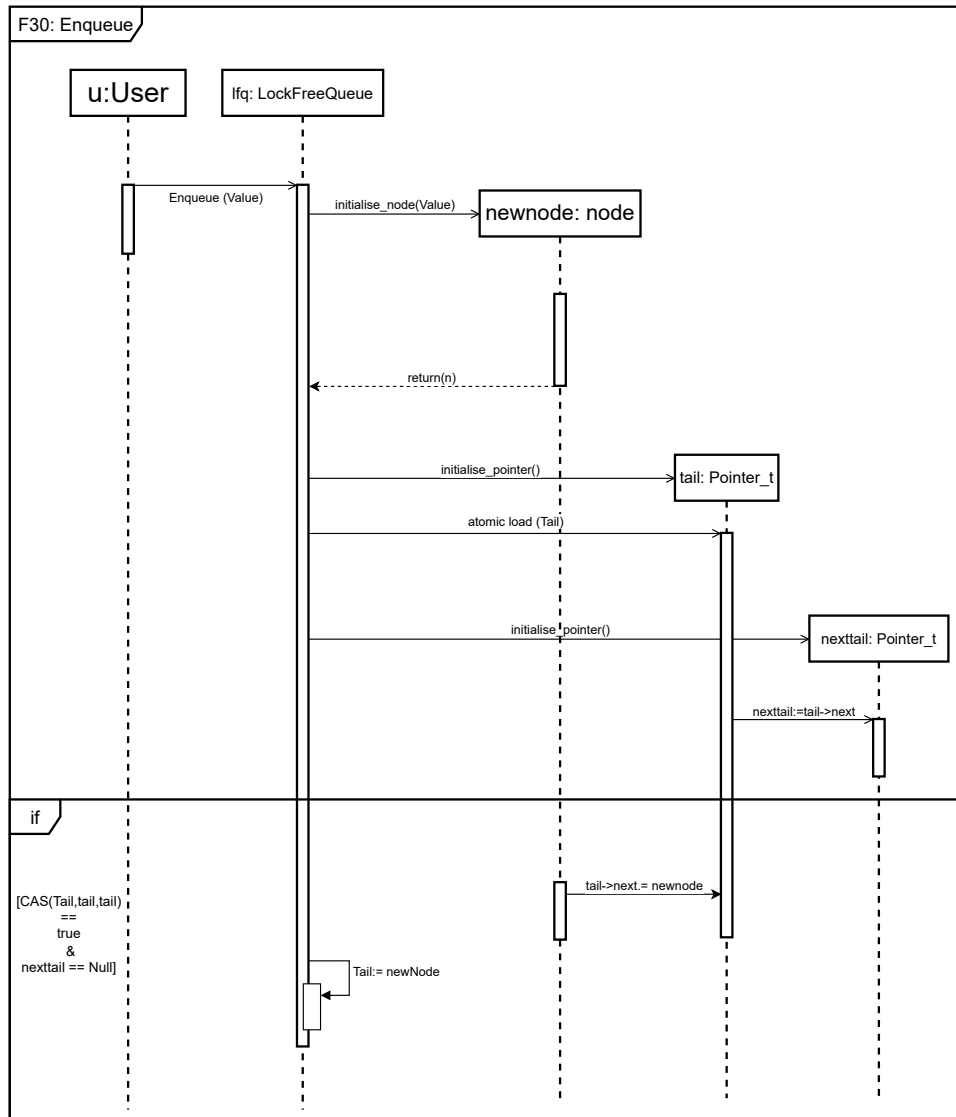


Abbildung 2.3: Enqueue

<F30>: In diesem Sequenzdiagramm, wird der Prozess Enqueue dargestellt. Dies geschieht, indem der User die Funktion enqueue(value) aufruft. Hinterher wird ein neuer Knoten newnode alloziert und dem der Wert value zugewiesen. Danach wird ein lokaler Zeiger tail initialisiert und dieser zeigt dann auf die aktuelle Adresse des globalen Zeigers Tail. Nachfolgend wird ein anderer Zeiger nexttail initialisiert und auf der Adresse von next von tail zeigen. Dann wird geprüft ob tail noch gleich zum globalen Tail ist und ob nexttail gleich NULL ist. Falls ja, dann werden die next von tail zu newnode und der globale Tail durch newnode ersetzt. Ansonsten wird der loop ab dem atomicload wiederholt bis enqueue erledigt ist.

2.4 Analyse von Funktionalität <F40>: <Dequeue>

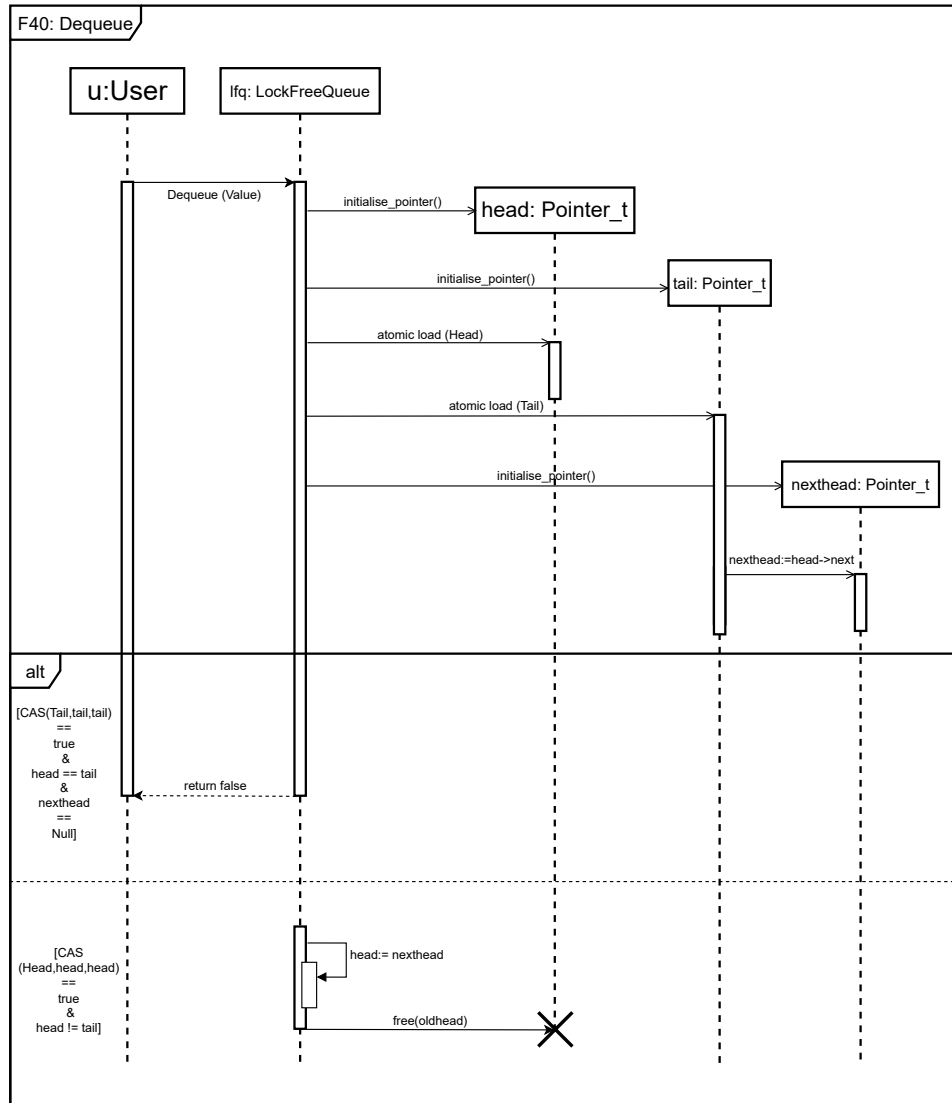


Abbildung 2.4: Dequeue

<F40>: In diesem Sequenzdiagramm, wird der Prozess des Dequeue dargestellt. Dies geschieht, indem der User die Funktion dequeue aufruft. Hinterher werden die Zeiger head, tail und nexthead initialisiert. Danach wird im head der Wert des globalen Heads, im tail der Wert des globalen Tails gelesen und dann im nexthead wird der Wert von head->next gelesen. Nachfolgend, falls head noch konsistent ist, also head und tail sind gleich, nexthead NULL ist, dann ist die Queue leer und die funktion gibt false zurück. Ansonsten wird der globale Head durch nexthead ersetzt und endlich wird der alte head freigegeben.

2.5 Analyse von Funktionalität <F50>: <Insert>

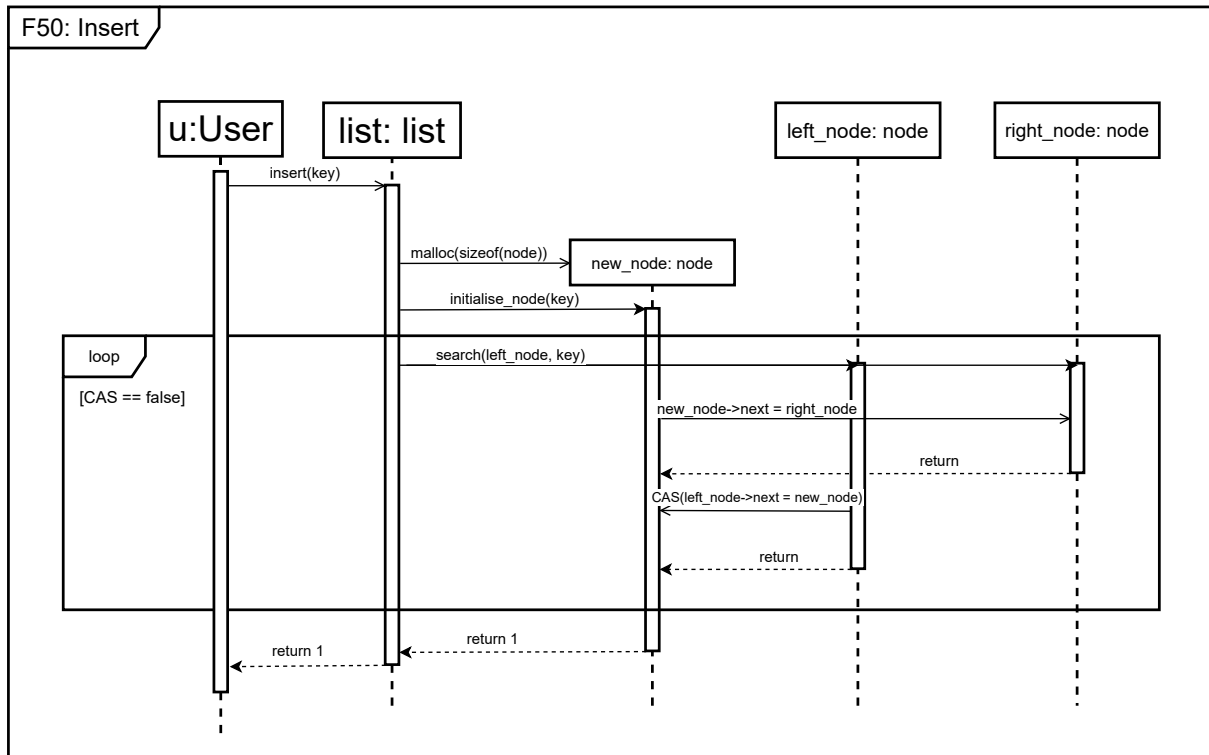


Abbildung 2.5: Insert im Set

<F50>: Für die Insertfunktion muss die Liste initialisiert vorliegen. Der User ruft dann die Funktion insert auf, die eine Node mitgegeben Key in das Set an die richtige Stelle einfügt. Im ersten Schritt wird eine neue Node alloziert und initialisiert. Danach begibt sich die Funktion in einen Loop, der läuft bis der CAS gelingt, darauf wird gleich noch genauer eingegangen. Zuerst wird mit Search die linke und rechte Node zum passenden Key herausgesucht und geladen. Danach wird newnode->next, die folgende node zu newnode die momentan noch auf NULL zeigt, auf rightnode gesetzt und rightnode wird fortan nicht mehr benötigt. Mit CAS wird hier der Compare-and-Swap gemeint der in 1.1 beispielsweise schon genauer beleuchtet wurde. Mit diesem wird leftnode->next atomar gesetzt und die Node, falls erfolgreich, ist nun eingefügt. Die leftnode wird jetzt auch nicht mehr benötigt und muss in einer neuen iteration neu geladen werden. Falls der CAS fehlschlägt, beginnt der Loop von vorne. Wenn der CAS erfolgreich ist, wird 1 returned und die sowohl die newnode als auch die Liste ist nicht mehr in benutzung.

2.6 Analyse von Funktionalität <F60>: <Delete>

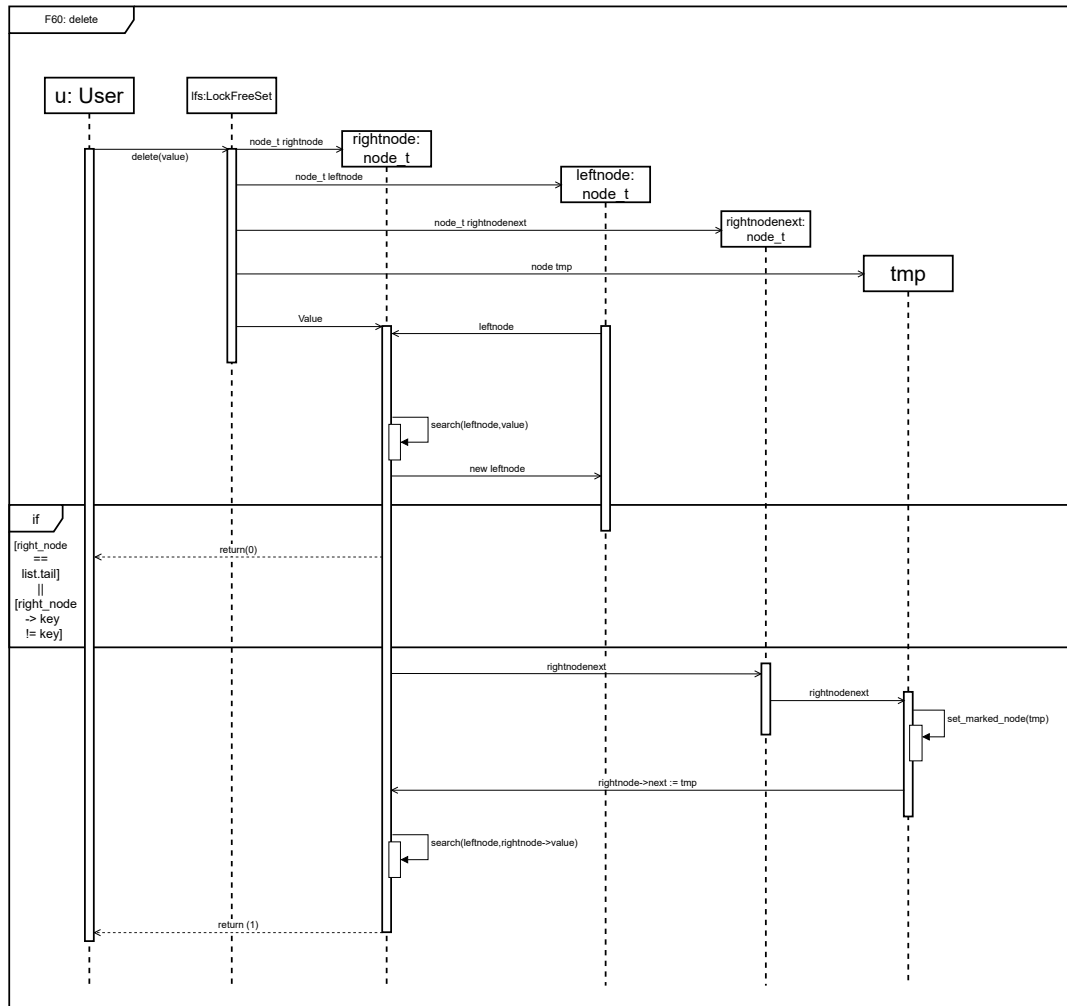


Abbildung 2.6: Delete

<F60>: In diesem Sequenzdiagramm, wird der Prozess des delete in Set dargestellt. Dies geschieht, indem der User die Funktion delete(value) aufruft. Hinterher werden lokale Objekte rightnode, rightnodenext, leftnode und tmp erstellt, um die alten und die neuen Nodes zu halten. Danach wird die Funktion Search in Set, die unten erklärt ist, auf rightnode mit den leftnode und gegebene value durchgeführt. Darauf wird, falls der Value von rightnode unterschiedlich ist, zu dem gesuchten value oder falls rightnode gleich tail der Liste ist, dann gibt die Funktion 0 zurück. Denn dann hat search keine passende Position für unseren Value im Set gefunden. Falls nicht, dann wird rightnodenext die next value von rightnode bekommen. tmp wird dann auf rightnode gesetzt, tmp wird als logisch gelöscht markiert und danach wird rightnodenext auf tmp gesetzt. Anschließend wird rightnode physisch gelöscht mithilfe von der search Funktion. Letztendlich wird der reference counter dekrementiert und rightnode freigegeben.

2.7 Analyse von Funktionalität <F70>: <Find>

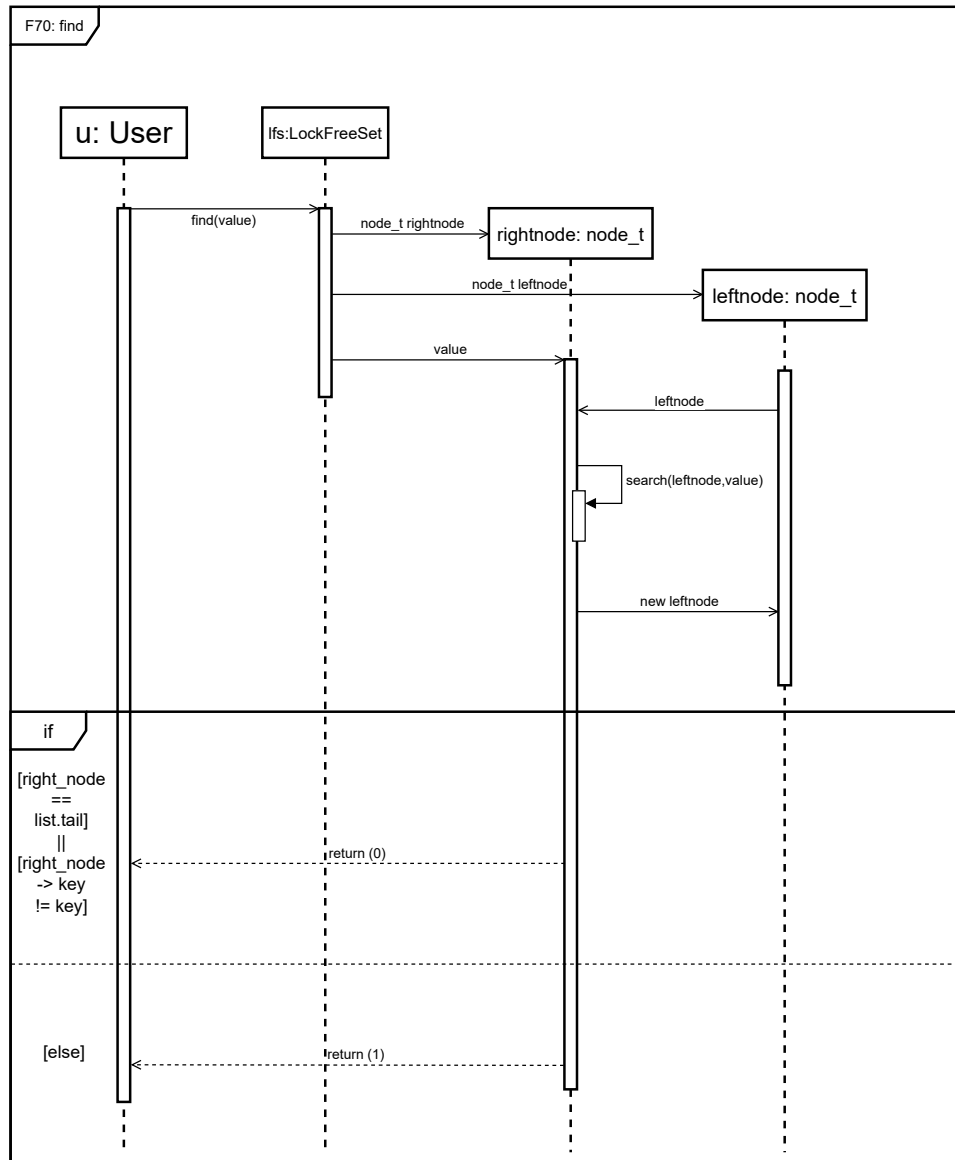


Abbildung 2.7: Find

<F70>: In diesem Sequenzdiagramm, wird der Prozess des Find in Set dargestellt. Dies geschieht, indem der User die Funktion `find(value)` aufruft. Hinterher werden die lokalen Objekte `leftnode` und `rightnode` erstellt. Danach wird die Funktion `Search` in Set, die unten erklärt ist, auf `rightnode` mit den `leftnode` und gegebene `value` durchgeführt. Darauf wird kontrolliert ob `Search` eine Valide Position bestimmt hat und dementsprechend 0 oder 1 returned.

2.8 Analyse von Funktionalität <F80>: <Search>

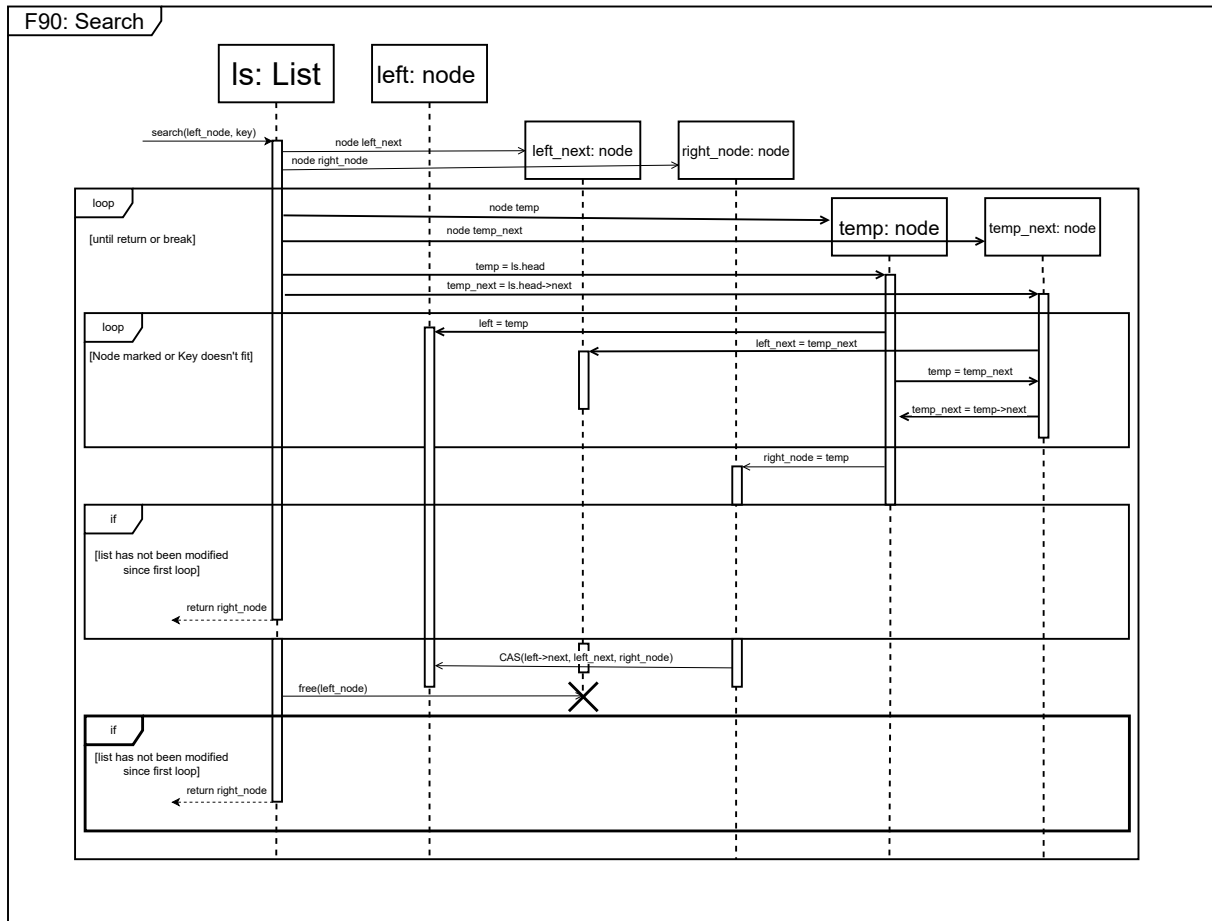


Abbildung 2.8: Search in Set

<F80>: Der Search Algorithmus lädt in 3 verschiedenen Abschnitten verschiedene Objekte. Zuerst werden leftnext und rightnode angelegt, danach begibt sich die Funktion in einen Loop der läuft bis ein return gegeben wird. Im ersten Teil des Loops werden zwei helper Nodes temp und tempnext angelegt und dann auf den Anfang der Liste gesetzt. Diese temporären Nodes sind für das Iterieren durch die Liste gedacht, was im nächsten Loop geschieht. Innerhalb des Loops werden left und leftnext auf die temporären Nodes gelegt, solange bis der Key einer unmarkierten temporären Node größer ist als der zu suchende Key. Nach dem Loop wird rightnode auf temp gesetzt, ab jetzt werde temp und tempnext nicht mehr benötigt. Falls weiterhin keine markierte Node, zwischen den beiden liegt, kann rightnode returned werden. Falls zwischendurch doch Nodes zwischen left und right markiert wurden, wird mithilfe eines CAS die betroffene Node ausgehängt und anschließend vom Speicher entfernt. Letztendlich wird nochmals überprüft, ob keine Veränderung in der Liste stattgefunden hat und falls ja, wird wieder die rightnode returned.

2.9 Analyse von Funktionalität <F90>: <Visualisieren>

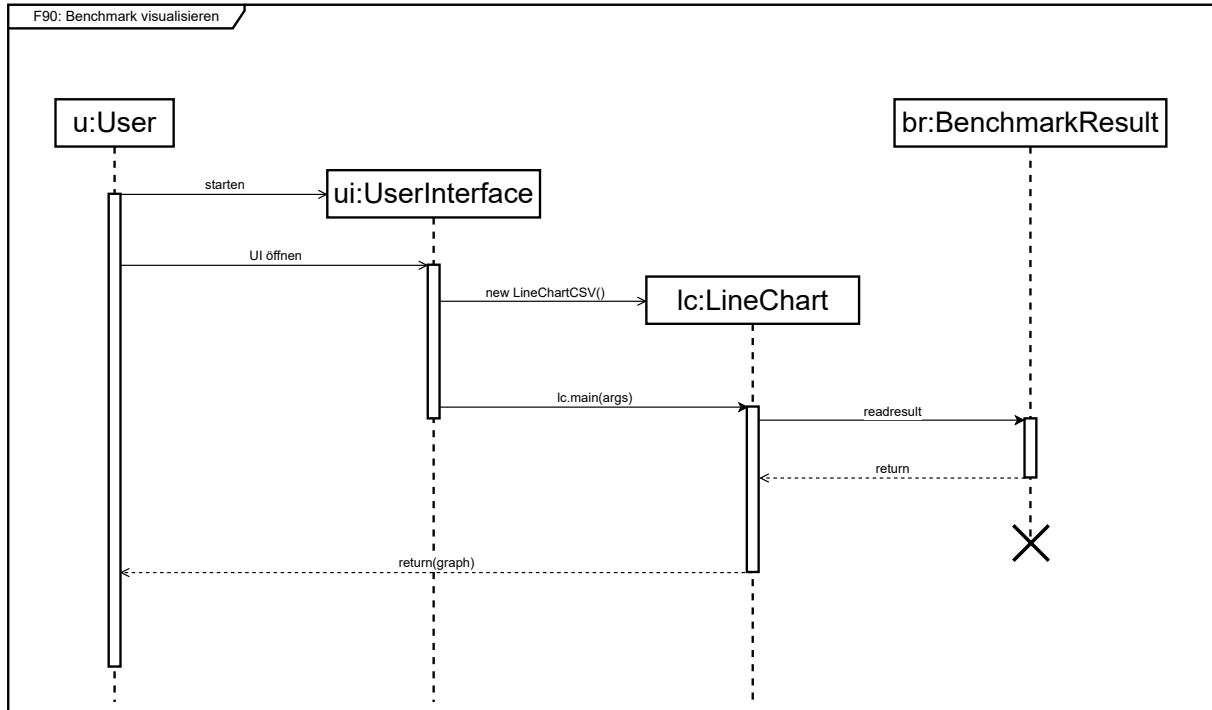


Abbildung 2.9: Visualisieren

<F90>: In diesem Sequenzdiagramm, wird der Prozess der Visualisierung dargestellt. Dies geschieht, indem der User das UserInterface startet. Das UserInterface erstellt dann ein neues Objekt der Klasse LineChart, welches unser Diagramm generiert. LineChart öffnet dann die .csv Datei mit den Benchmark results, die dann returned werden und generiert mit den Daten den gewünschten Graphen, der dann dem User returned wird.

2.10 Analyse von Funktionalität <F100>: <Benchmark>

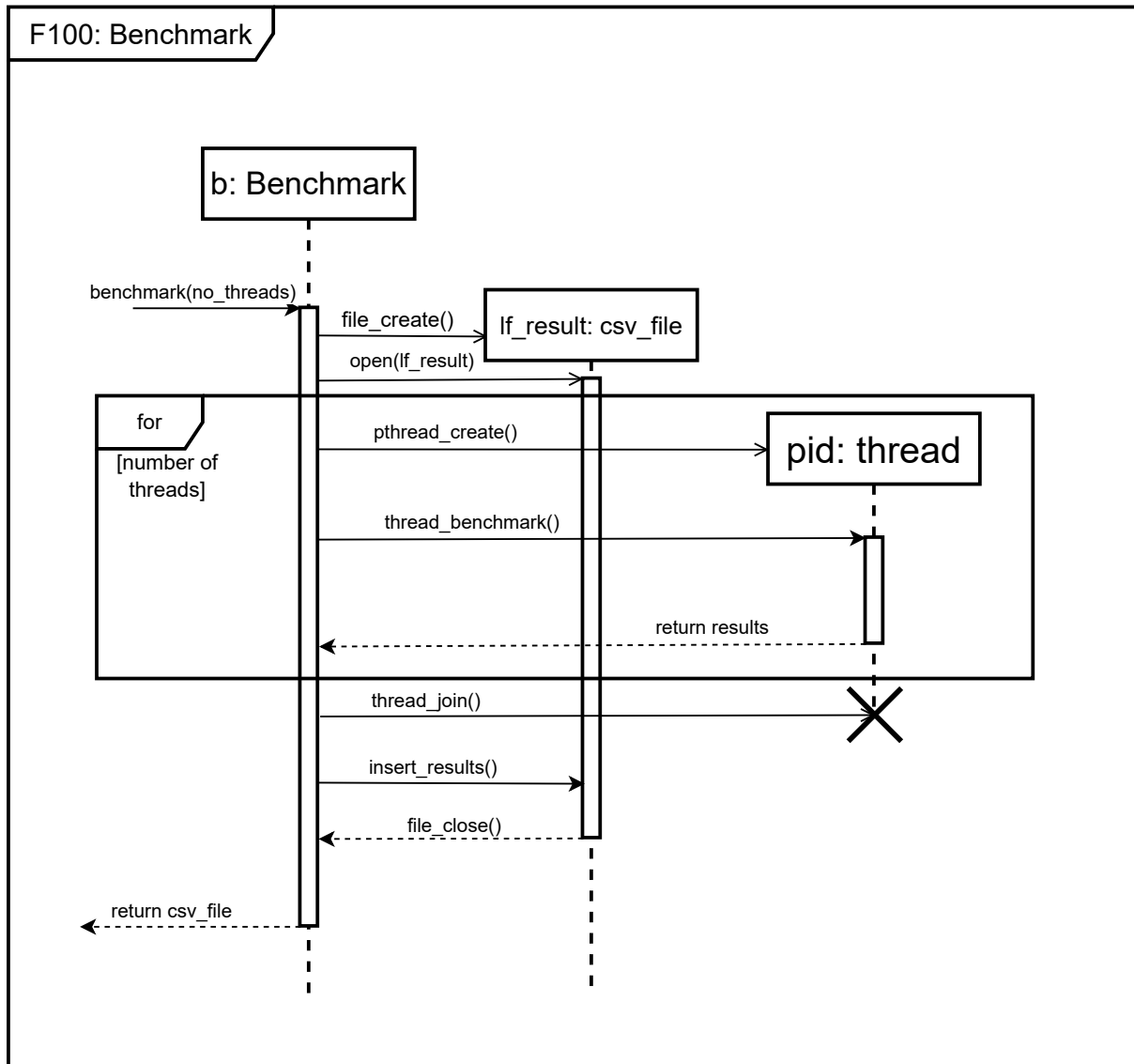


Abbildung 2.10: Benchmark

<F100>: Bei dem Benchmark wird eine .csv Datei angelegt und geöffnet, die dann die Ergebnisse beinhalten soll. Die gewünschte Anzahl an Threads werden angelegt und dann mit der Benchmark Funktion aufgerufen. Diese Funktion berechnet die Gesamtzeit und die Latenz für die gewünschte Anzahl der Operationen. Sobald die Threads mit der ermittlung fertig sind, werden die Ergebnisse returned und die Threads werden aufgelöst. Anschließend werden die Ergebnisse in die Datei geschrieben und diese dann geschlossen. Schlussendlich returned der Benchmark die Datei für die Visualisierung.

3 Resultierende Softwarearchitektur

Dieser Abschnitt hat die Aufgabe, einen Überblick über die zu entwickelnden Komponenten und Subsysteme zu liefern.

3.1 Komponenten-Spezifikation

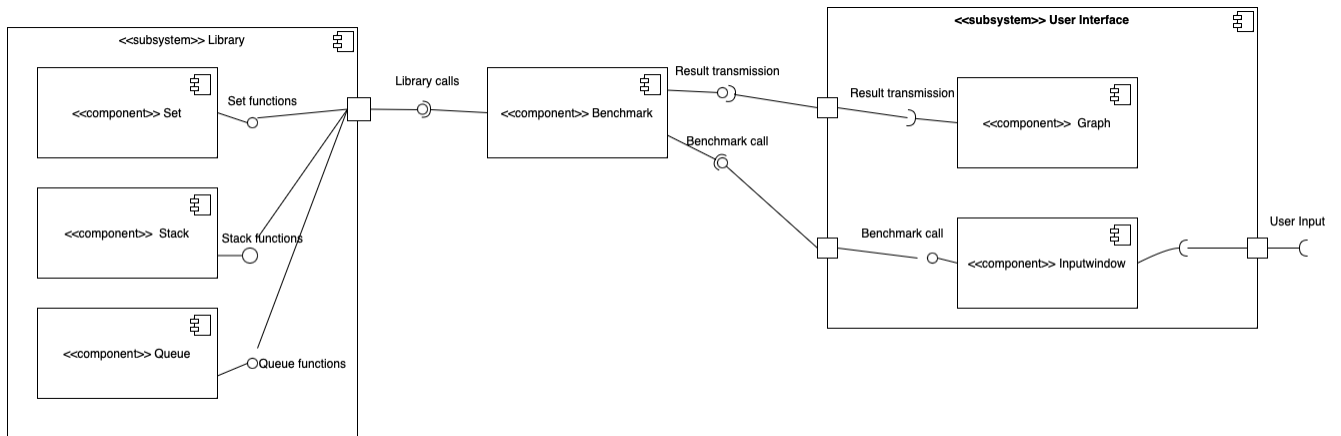


Abbildung 3.1: Komponentendiagramm für Library und UI

Das Subsystem Library enthält die Komponenten der lockfreien Datenstrukturen mit ihren Funktionen, jede Datenstruktur mit ihren eigenen Functioncalls und deshalb eigenen Interfaces. Das Subsystem User Interface setzt sich aus einem Inputwindow und dem Ergebnisgraphen zusammen. Auf die Komponenten innerhalb der Subsysteme wird im folgenden eingegangen.

Komponente <C10>: <Set>

Diese Komponente ist Teil des Subsystems 'Library' und beinhaltet alle Funktionalitäten rund um die Datenstruktur Set beziehungsweise List in ihrer lockfreien Variante. Der Benchmark kann über das Interface 'Library calls' die Funktionen nutzen.

Komponente <C11>: <Queue>

Diese Komponente ist Teil des Subsystems 'Library' und beinhaltet alle Funktionalitäten rund um die Datenstruktur Queue in ihrer lockfreien Variante. Der Benchmark kann über das Interface 'Library calls' die Funktionen nutzen.

Komponente $\langle C12 \rangle$: $\langle \text{Stack} \rangle$

Diese Komponente ist Teil des Subsystems 'Library' und beinhaltet alle Funktionalitäten rund um die Datenstruktur Stack in ihrer lockfreien Variante. Im Benchmark kann über das Interface 'Library calls' die Funktionen aufgerufen werden.

Komponente $\langle C20 \rangle$: $\langle \text{Benchmark} \rangle$

Die Benchmark Komponente berechnet die overall Performance sowie die Latenz der einzelnen Operationen für die gewünschte Datenstruktur. Sie implementiert über das Interface 'Library calls' die benötigten Operationen der Datenstrukturen. Ihre Parameter bekommt Sie über das Interface 'Benchmark call', in dem der Benchmark mit den gewünschten Werten aufgerufen wird. Nachdem die Performance ermittelt wurde, übergibt der Benchmark die Ergebnisse an die Graf-Komponente, mithilfe des Interface 'Result transmission'.

Komponente $\langle C30 \rangle$: $\langle \text{Inputwindow} \rangle$

Die Komponente Input window ist Teil des Userinterface Subsystems und bietet dem User die Möglichkeit die gewünschten Benchmark-Parameter sowie die gewollte Datenstruktur festzulegen. Die Parameter werden dann zusammen mit dem Benchmark call über das Interface 'Benchmark call' weitergegeben. Der User kann die Daten mithilfe des 'User Input' Interface eingeben.

Komponente $\langle C31 \rangle$: $\langle \text{Graph} \rangle$

Diese Komponente ist Teil des Userinterface Subsystems und stellt die grafische Darstellung der Benchmark Ergebnisse bereit. Damit die Komponente weiß, wann der Benchmark fertig ist und welche Ergebnisse er liefert implementiert sie das Interface 'Result transmission' bereit.

3.2 Schnittstellenspezifikation

Im Folgenden werden die einzelnen Schnittstellen der Komponenten aus der Komponentenspezifikation näher erläutert, d.h. die von ihnen zur Verfügung gestellten Operationen werden dokumentiert. Die Tabelle ist dabei um so viele Zeilen zu erweitern, wie es Schnittstellen im Komponenten-diagramm gibt. In der innen liegenden Aufteilung ist für jede Operation einer Schnittstelle eine Zeile einzufügen. Reine Set- und Get-Aufrufe brauchen nicht aufgeführt zu werden (sollten auch möglichst nicht komponentenübergreifend auftauchen).

Schnittstelle $\langle I10 \rangle$: $\langle \text{Set functions} \rangle$

Da es sich hier um Library calls handelt werden C function calls genutzt die vom Benchmark importiert wurden. Alle nachfolgenden Funktionen sind der Form `function(int key, Set set)` wobei `function` durch den Namen der Operation zu ersetzen ist.

Operation	Beschreibung
insert	Insert fügt eine Node mit gewünschten Key in das mitgegebene Set ein.
delete	Delete löscht die Node mit gewünschtem Key aus dem Set
find	Find sucht im Set nach einer Node mit dem mitgegebenen Key und gibt 1 oder 0 zurück je nach ergebnis der Suche.

Schnittstelle $\langle I11 \rangle$: **<Queue functions>**

Da es sich hier um Library calls handelt werden C function calls genutzt die vom Benchmark importiert wurden. Alle nachfolgenden Funktionen sind der Form `function(int key, Queue q)` wobei `function` durch den Namen der Operation zu ersetzen ist.

Operation	Beschreibung
enqueue	Enqueue fügt eine Node mit gewünschten Key am Ende der mitgegebene Queue ein.
dequeue	Dequeue löscht die Node am anfang der mitgegebenen Queue aus der Datenstruktur.

Schnittstelle $\langle I12 \rangle$: **<Stack functions>**

Da es sich hier um Library calls handelt werden C function calls genutzt die vom Benchmark importiert wurden. Alle nachfolgenden Funktionen sind der Form `function(int key, Stack s)`, wobei `function` durch den Namen der Operation zu ersetzen ist.

Operation	Beschreibung
Push	Push fügt eine Node mit gewünschten Key am anfang des mitgegebenen Stacks ein.
pop	Pop löscht die Node am anfang des mitgebenden Stacks aus der Datenstruktur.

Schnittstelle $\langle I20 \rangle$: **<Library calls>**

Das Library calls Interface gibt dem Benchmark die Möglichkeit auf die functions der einzelnen Datenstrukturen zuzugreifen mithilfe der darüber aufgeführten internen Schnittstellen.

Da es sich hier um Library calls handelt, werden C function calls genutzt die vom Benchmark importiert wurden. Alle nachfolgenden Funktionen sind der Form `function(int key, Stack s)` wobei `function` durch den Namen der Operation zu ersetzen ist.

Operation	Beschreibung
insert	Insert fügt eine Node mit gewünschten Key in das mitgegebene Set ein.
delete	Delete löscht die Node mit gewünschtem Key aus dem Set
find	Find sucht im Set nach einer Node mit dem mitgegebenen Key und gibt 1 oder 0 zurück je nach ergebnis der Suche.
enqueue	Enqueue fügt eine Node mit gewünschten Key am Ende der mitgegebene Queue ein.
dequeue	Dequeue löscht die Node am anfang der mitgegebenen Queue aus der Datenstruktur.
push	Push fügt eine Node mit gewünschten Key am anfang des mitgegebenen Stacks ein.
pop	Pop löscht die Node am anfang des mitgegebenen Stacks aus der Datenstruktur.

Schnittstelle $\langle I21 \rangle$: **<Benchmark call>**

Die Interfaces 'Benchmark call' innerhalb und außerhalb des Subsystems User Interface sind identisch zu betrachten, da 'Benchmark call' außerhalb nur eine Weiterleitung außerhalb des Subsystems darstellt.

Die Parameter i, d und ds sind die Parameter für den Benchmark die vom Inputwindow geliefert worden und sind dementsprechend zu ersetzen. Weitere Erläuterungen zu den Parametern sind innerhalb der Beschreibung.

Operation	Beschreibung
./benchmark i d ds	Aufruf des C Benchmarks mit den Parametern j i, den Anzahl der Inputs, d, Anzahl der Deletions und ds, der gewünschten Datenstruktur.

Schnittstelle $\langle I22 \rangle$: **<Result transmission>**

Die Interfaces 'Result transmission' innerhalb und außerhalb des Subsystems User Interface sind identisch zu betrachten, da 'Result transmission' außerhalb nur eine Weiterleitung außerhalb des Subsystems darstellt.

Operation	Beschreibung
transmitResult	Aufruf des C Sockets, der der Graph komponente mitteilen soll, dass der Benchmark fertig ist und die Ergebnisse aufgerufen werden können.

Schnittstelle $\langle I30 \rangle$: **<User Input>**

User input stellt eine Schnittstelle bereit in der User die gewünschten Parameter in das bereitgestellte Fenster eintragen kann, es findet daher kein Funktionsaufruf im klassischen Sinne statt.

3.3 Protokolle für die Benutzung der Komponenten

Die einzigen Komponenten die zur Wiederverwendung geeignet/gewollt sind, sind die Komponenten der Library, also die Datenstrukturen. Um die korrekte Verwendung der Datenstrukturen sicherzustellen, folgen nun Protokoll-Statechart's die aufzeigen wie die Datenstrukturen korrekt zu verwenden sind. Da alle Datenstrukturen demselben Protokoll zur verwendung folgen gibt es eine allgemeingültige Version für Stack, Set und Queue.

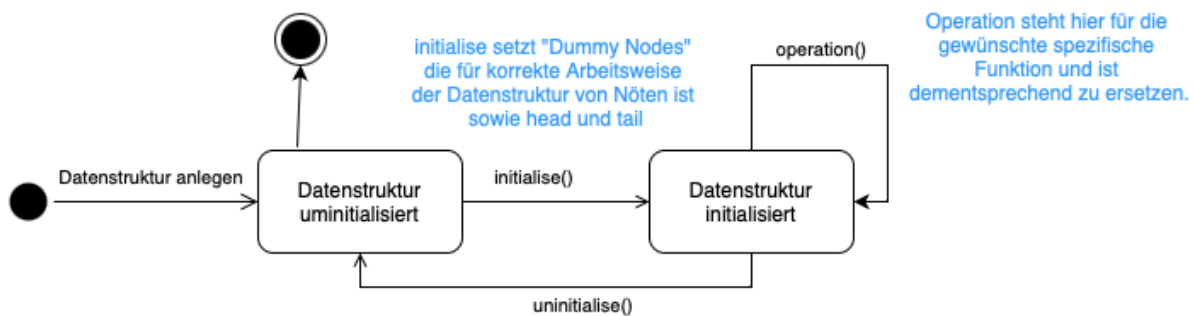


Abbildung 3.2: Statechart für die korrekte Verwendung der Datenstrukturen

4 Verteilungsentwurf

In diesem Kapitel wird die verteilte Anwendung des Softwareprojektes einer C-Library für lockfreie Datenstrukturen beschrieben. Die Verteilung auf verschiedene Geräte, beschränkt sich dabei auf ihren jeweiligen Computer, da sich dort alles beendet, was sie benötigen. Dort arbeiten verschieden von uns entwickelte Programme zusammen.

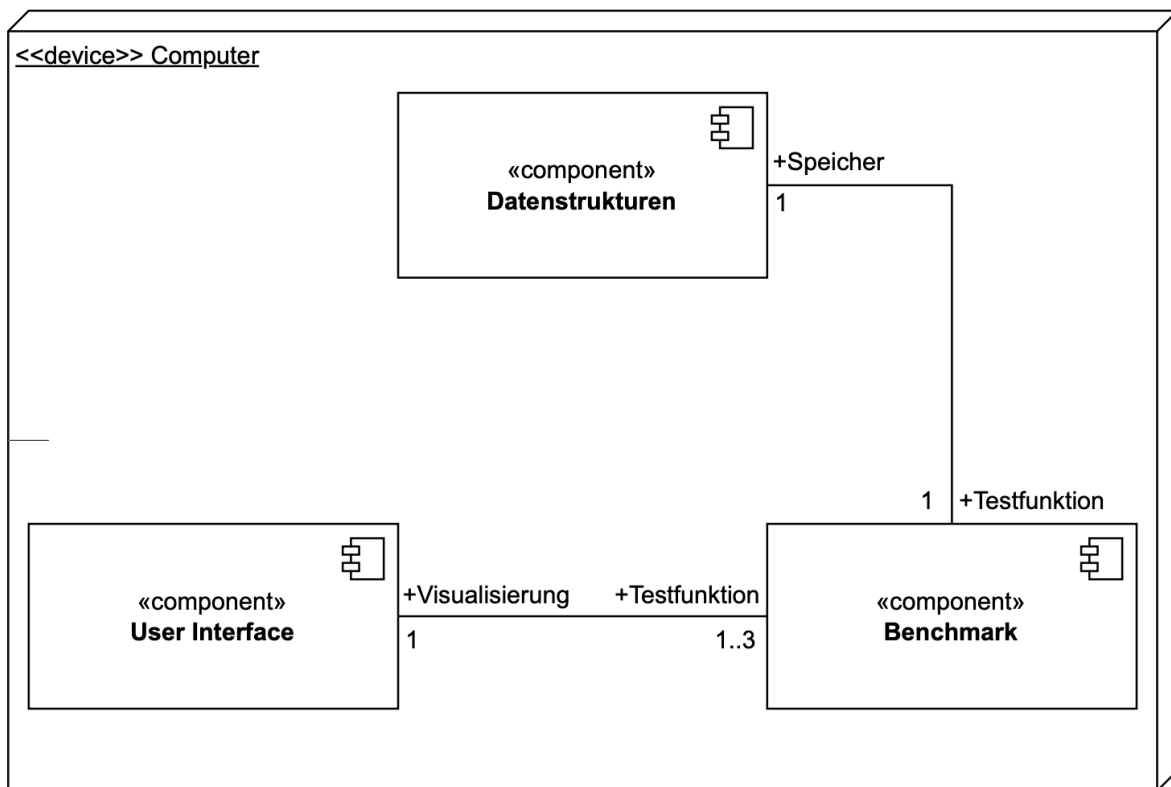


Abbildung 4.1: Verteilungsdiagramm

5 Implementierungsentwurf

Dieser Abschnitt hat die Aufgabe, alle verwendeten Klassen und Bibliotheken zu dokumentieren. Dabei wird jede Komponente aus Kapitel 3 gesondert betrachtet.

5.1 Implementierung von Komponente <10>: <Set>:

5.1.1 Paket-/Klassendiagramm

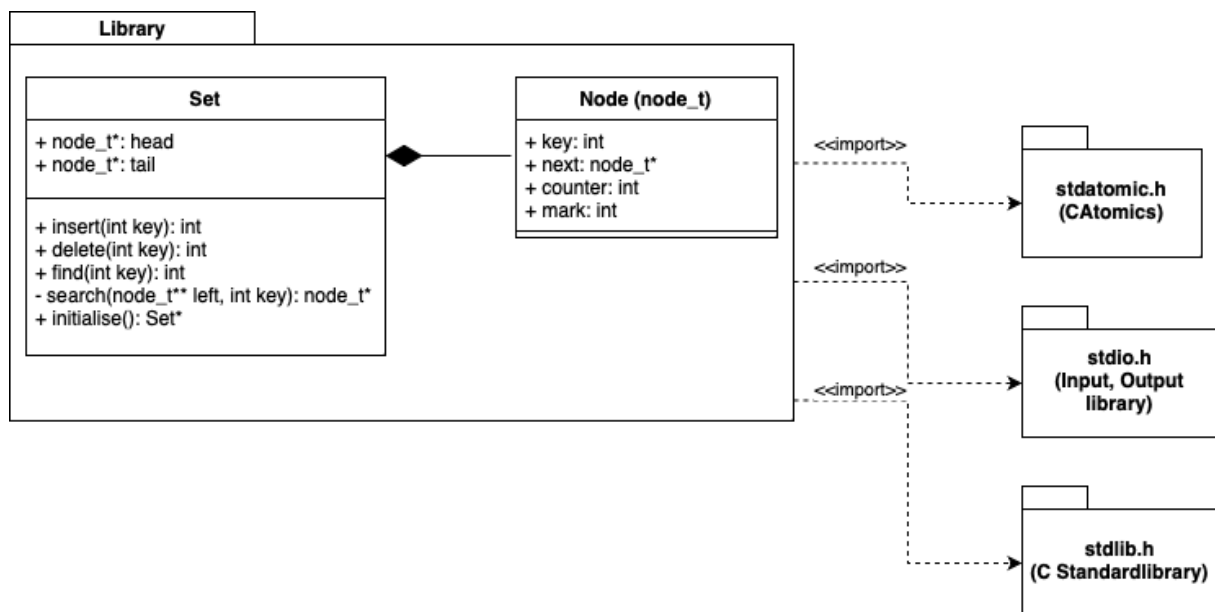


Abbildung 5.1: Klassendiagramm Set (C10)

5.1.2 Erläuterung

Set $\langle CL10 \rangle$

Aufgabe

Datenstruktur Set in lockfreier Variante, sortierte Liste an int values die eingefügt oder entfernt werden können.

Attribute

Nodes die aus einem next pointer, einem key, einem mark sowie counter bestehen. Mark ist notwendig für den Algorithms um Nodes zu markieren, counter wird für memory allocation verwendet.

Zwei 'dummy' Node Pointer die auf Anfang und ende der Liste Zeigen.

Operationen

insert(int key): Insert fügt eine Node mit gewünschten Key in das mitgegebene Set ein.

delete(int key): Delete löscht die Node mit gewünschtem Key aus dem Set.

find(int key): Find sucht im Set nach einer Node mit dem mitgebenden Key und gibt 1 oder 0 zurück je nach ergebnis der Suche.

initialise(): Initialise initialisiert das Set mit dummy Nodes, sodass es verwendet werden kann.

search(nodet** left, int key): Search sucht nach einer passenden Stelle für den gewünschten Key, gibt die rechte Node als Pointer zurück und speichert die linke im Parameter. Diese Funktion ist private und sollte nur von Delete und Insert aufgerufen werden.

Kommunikationspartner

Kein Kommunikationspartner vorhanden.

5.2 Implementierung von Komponente <11>: <Queue>:

5.2.1 Paket-/Klassendiagramm

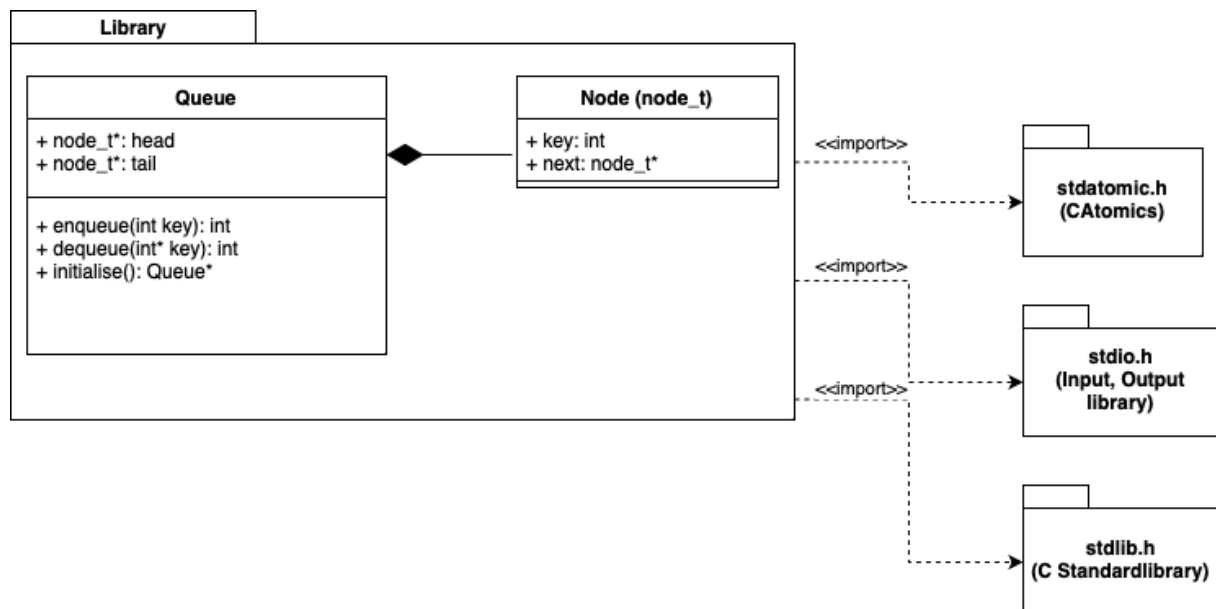


Abbildung 5.2: Klassendiagramm Queue <C11>

5.2.2 Erläuterung

Queue<CL11>

Aufgabe

Datenstruktur Queue in lockfreier Variante, FIFO Queue an int values die eingefügt oder entfernt werden können.

Attribute

Nodes die aus einem next pointer und einem key bestehen. Zwei 'dummy' Node Pointer die auf Anfang und ende der Queue Zeigen.

Operationen

`enqueue(int key)`: Hängt eine Node mit gewünschter Value an das Ende der Queue an.

`dequeue(int key)`: Entfernt die vorderste 'head' node aus der Queue und gibt ihren Wert aus.

`initialise()`: Initialise initialisiert die Queue mit dummy Nodes, sodass sie verwendet werden kann.

Kommunikationspartner

Keine Kommunikationspartner vorhanden.

5.3 Implementierung von Komponente <12>: <Stack>:

5.3.1 Paket-/Klassendiagramm

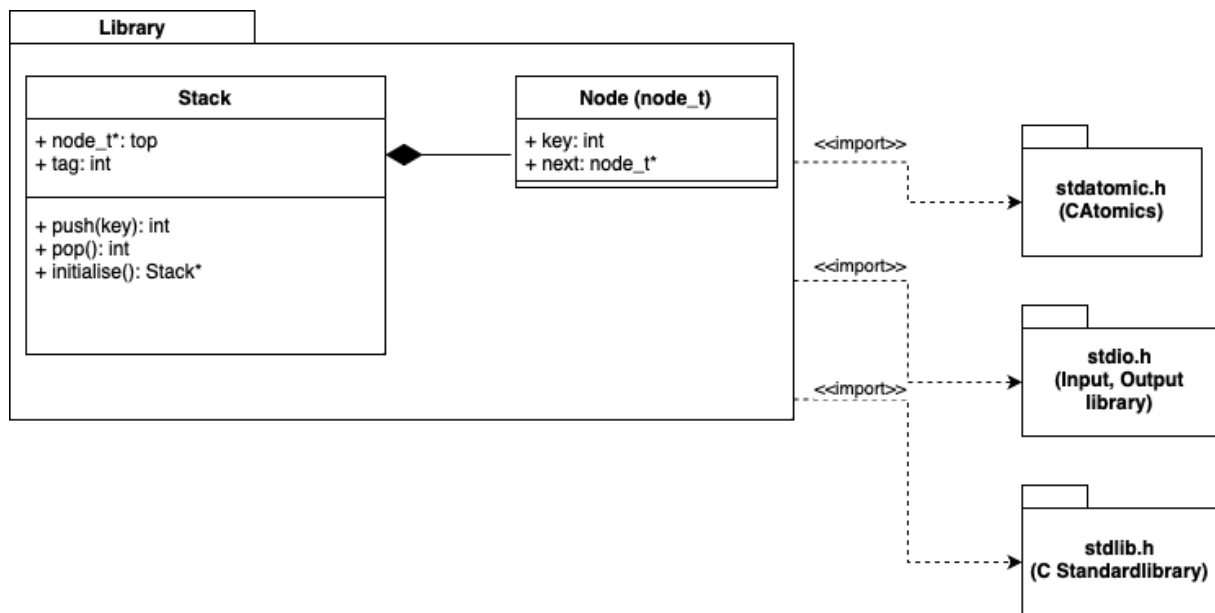


Abbildung 5.3: Klassendiagramm Stack (C12)

5.3.2 Erläuterung

Stack<CL12>

Aufgabe

Datenstruktur Queue in lockfreier Variante, LIFO Stack an int values die eingefügt oder entfernt werden können.

Attribute

Nodes die aus einem next pointer und einem key bestehen. Ein node pointer top der auf eine 'dummy' node zeigt, sowie ein Tag der das ABA Problem löst.

Operationen

push(int key): Hängt eine Node mit gewünschter Value an den Anfang des Stacks an.

pop(int key): Entfernt die vorderste 'head' node aus dem Stack und gibt ihren Wert aus.

`initialise()`: Initialise initialisiert den Stack mit einer dummy Node, sodass er verwendet werden kann.

Kommunikationspartner

Kein Kommunikationspartner vorhanden.

5.4 Implementierung von Komponente <20>: <Benchmark>:

5.4.1 Paket-/Klassendiagramm

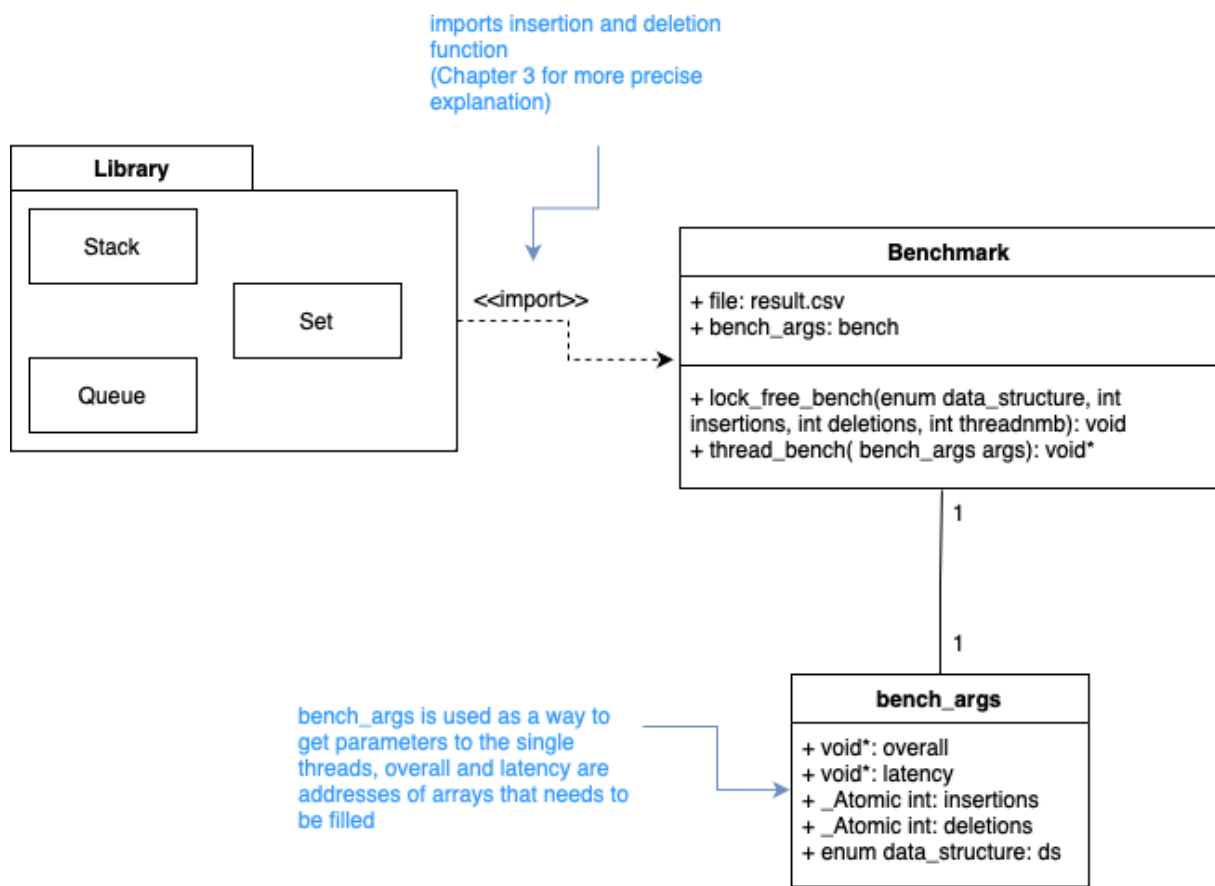


Abbildung 5.4: Klassendiagramm Benchmark <C20>

5.4.2 Erläuterung

Benchmark $\langle CL20 \rangle$

Aufgabe

Die Benchmark Komponente berechnet die overall Performance sowie die Latenz der einzelnen Operationen für die gewünschte Datenstruktur. Nachdem die Performance ermittelt wurde übergibt der Benchmark die Ergebnisse an die Graph Komponente.

Attribute

Der Benchmark enthält die File result.csv und die Argumente benchargs. Benchargs enthält zwei Pointer auf Array Adressen, die Anzahl der Insertions und Deletions sowie die gewünschte Datenstruktur.

Operationen

lockfreebench(enum datastructure, int insertions, int deletions, int threadnmb): Diese Operation erstellt die Threads mit Startoperation , die File results.csv und die Argumente benchargs.

threadbench(benchargs): Diese Operation testet die einzelnen Threads auf ihre Performance basierend auf den Argumenten die in benchargs zu finden sind.

Kommunikationspartner

Inputwindow für den Aufruf und Parameterübergabe und Graph zur übergabe der Benchmark-Ergebnisse.

5.5 Implementierung von Komponente <30>: <Inputwindow>:

5.5.1 Paket-/Klassendiagramm

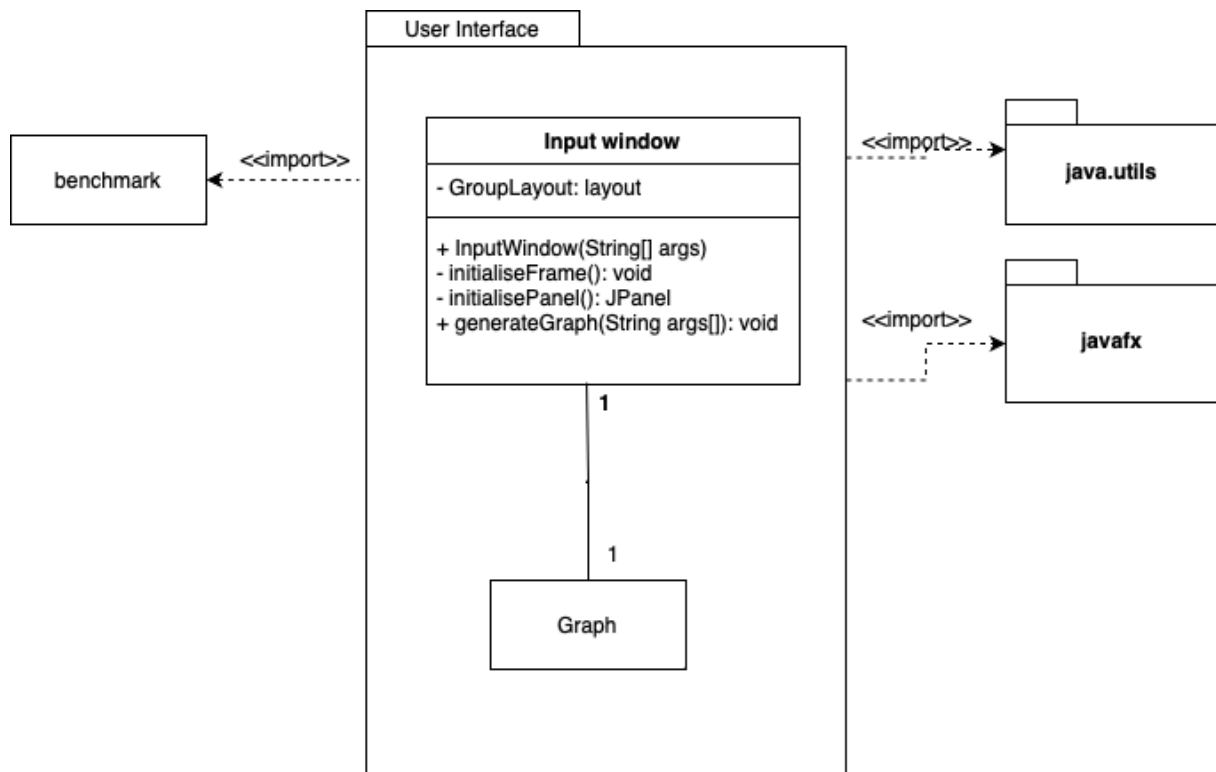


Abbildung 5.5: Klassendiagramm Inputwindow <C30>

5.5.2 Erläuterung

Inputwindow<CL30>

Aufgabe

Inputwindow bietet dem User einen Weg die gewünschten Benchmark Parameter einzutragen, danach wird der Benchmark von Inputwindow aufgerufen.

Attribute

Inputwindow enthält lediglich ein GroupLayout in welchem angegeben ist wie die Eingabefelder positioniert sein sollen.

Operationen

inputWindow(String[] args): Konstruktor für das InputWindow in welchem die Eingabefelder generiert und positioniert werden.

`initialiseFrame()`: Helfer funktion die das `JFrame` initialisiert und zur weiterverwendung vorbereitet.

`initialisePanel()`: Helfer funktion die das `JPanel` initialisiert und zur weiterverwendung vorbereitet.

`generateGraph(String[] args)`: Funktion zum erstellen eines Graph Fensters sowie Aufruf des Benchmarks.

Kommunikationspartner

`Inputwindow` kommuniziert über einen functioncall mit dem `Benchmark` und ruft die `Graph` Komponente auf.

5.6 Implementierung von Komponente <31>: <Graph>:

5.6.1 Paket-/Klassendiagramm

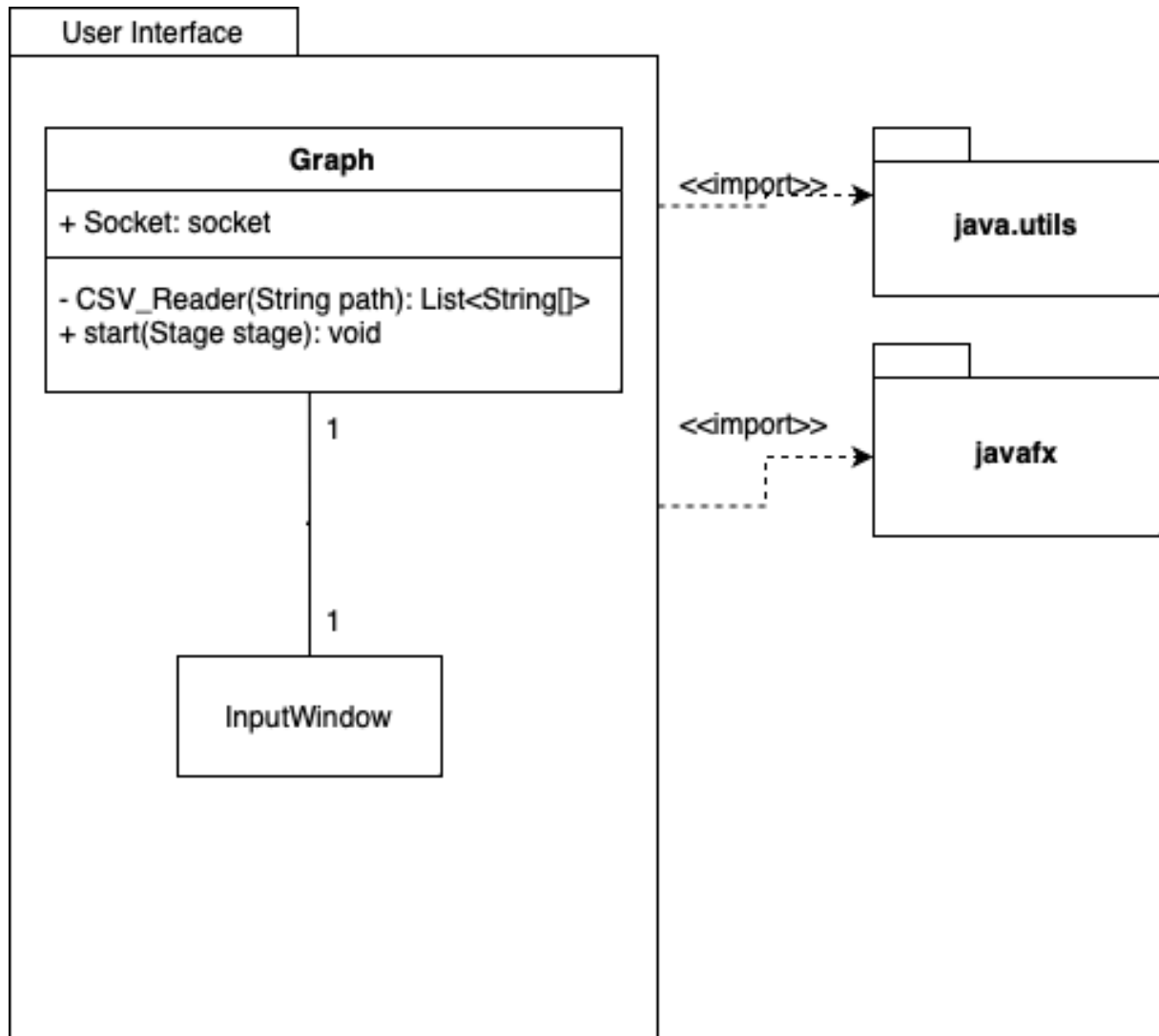


Abbildung 5.6: Klassendiagramm Graph <C31>

5.6.2 Erläuterung

Graph<CL31>

Aufgabe

Graph zeigt dem User die Performance des gewünschten Benchmarks in einem Liniengrafen.

Attribute

Der Graph enthält einen Socket über dem mit dem Benchmark kommuniziert wird.

Operationen

CSVReader(String path): Diese Operation liest die Werte aus der result.csv und speichert sie in eine Liste zur weiterverwendung.

start(Stage stage): Generiert den Graphen basierend auf den Daten der Liste aus dem CSVReader.

Kommunikationspartner

Graph kommuniziert mit dem Benchmark über eine Socket-Verbindung, über die mitgeteilt wird, wenn die result.csv geöffnet werden darf beziehungsweise ob Fehler aufgetreten sind.

6 Datenmodell

Dieses Kapitel bleibt leer, da wir keinerlei Daten dauerhaft speichern.

7 Konfiguration

In diesem Kapitel werden sämtliche Konfigurationen genannt, die für die Erfüllung des Softwareprojekts notwendig sind.

Das Projekt besteht dabei aus zwei Teilen, drei vollständig implementierten Datenstrukturen und deren Visualisierung.

Um die Verwendung der Datenstrukturen für möglichst viele Personen zu ermöglichen, benötigt der Computer lediglich eines der drei am häufigsten verwendeten Betriebssysteme, Windows, Mac OS oder LINUX. Es wird keine Internetverbindung benötigt um die Visualisierung oder die Library zu benutzen. Um die Library in ein eigenes Programm einzubinden, muss sie lediglich heruntergeladen und importiert werden, auch hier sind keine besonderen config Dateien nötig.

Zur Ausführung der Visualisierung wird eine Java Installation benötigt und muss gegebenenfalls vom Nutzer installiert werden. Für die Visualisierung muss auch gegebenenfalls JavaFX installiert und aufgesetzt werden.

8 Änderungen gegenüber Fachentwurf

Für bessere Memory allocation wird der 'Reference Counter' durch eine 'Free List' ersetzt. Es kommt zu ungewünschtem Verhalten, wenn auf eine Speicheradresse zugegriffen wird, nachdem sie bereits freigegeben wird. Um das zu verhindern, wird ein Free list benutzt. Jeder Thread hat da eigene Liste, die aus Speicheradressen von Nodes besteht. Falls eine Node aus der Datenstruktur entfernt wird, wird diese Speicheradresse in die Liste von Thread zum später benutzen hinzugefügt. Wenn ein Thread einen Node allozieren will, schaut er erst seine eigene Free-Liste. Für den Fall, dass die Liste nicht leer ist, benutzt der Thread eine Speicheradresse von der Liste. Wenn Liste leer ist, alloziert er mit malloc eine neue Speicheradresse.

9 Erfüllung der Kriterien

Nachfolgend wird beschrieben, wie die einzelnen Kriterien des Pflichtenheftes erfüllt werden und worauf geachtet wird. Es ist dabei explizit auf die definierten Kriterien des Pflichtenheftes zu verweisen. Das Kapitel sollte fortlaufend gepflegt werden. **Es sollte auch darauf verwiesen werden, welche Komponente das Kriterium erfüllt.**

9.1 Muss-Kriterien

Muss-Kriterien: unabdingbare Leistungen der Software.

Hier wird aufgeführt, welche Funktionalitäten/Leistungen das Softwareprodukt in jedem Fall erfüllen muss, damit es genutzt werden kann.

⟨RM1⟩ Die Datenstrukturen sind ausschließlich in der Programmiersprache C geschrieben. In der Library sind alle Datenstrukturen ausschließlich in C implementiert. Bis auf in C geschriebene Dateien befinden sich nur Hilfsdateien, wie CMakeLists.txt- oder .gitignore-Dateien, in den implementierten Datenstrukturen.

⟨RM2⟩ Es muss die atomic-Library verwendet werden.

Die `<stdatomic.h>`-Library wird in allen Datenstrukturen inkludiert und im Code genutzt.

⟨RM3⟩ Die Software muss lockfrei sein.

In der Library werden keine Locks aufgerufen. Objekte auf die verschiedene Threads zugreifen können werden mittels Funktionen aus der `<stdatomic.h>`-Library koordiniert. Wir verwenden in den Datenstrukturen hauptsächlich die CAS-Funktion (compare and swap).

⟨RM4⟩ Es dürfen keine Malloc-Fehlschläge, falsche Operationen bei leeren Datenstrukturen oder ungültige Eingaben beim Einfügen von Elementen in die Datenstrukturen auftreten.

⟨RM5⟩ Es dürfen keine Malloc-Fehlschläge, falsche Operationen bei leeren Datenstrukturen oder ungültige Eingaben beim Entfernen von Elementen aus den Datenstrukturen auftreten.

⟨RM6⟩ Es dürfen keine Malloc-Fehlschläge, falsche Operationen bei leeren Datenstrukturen oder ungültige Eingaben beim Finden von Elementen im Set auftreten.

Alle Funktionen in den Datenstrukturen werden, einzeln oder zeitgleich, umfangreich auf verschiedenen Hardwaresystemen getestet. Somit wird versucht, alle möglichen Szenarien zu adaptieren und auf die genannten Fehler zu prüfen. Auf den Betriebssystemen Linux und MacOS wird zusätzlich ein AddressSanitizer verwendet.

⟨RM7⟩ Die Software muss mit einer variablen aber festgelegten Anzahl von Threads ausführbar sein.

Die in RM4-6 beschriebenen Testfälle werden mit unterschiedlich vielen Threads und mit eingeschalteten ThreadSanitizer zusätzlich ausgeführt.

9.2 Soll-Kriterien

Soll-Kriterien: erstrebenswerte Leistungen

Dies sind Kriterien, die für die Lauffähigkeit des Produkts nicht zwingend erforderlich sind, für die Erreichung der Projektziele aber erfüllt werden sollten.

⟨RS1⟩ Als Datenstrukturen sollen Stack, Queue und List/Set implementiert werden.

Harris List, Queue und Stack sind in der Library implementiert.

⟨RS2⟩ Als Compiler sollte clang verwendet werden.

Auf den Betriebssystemen Linux, MacOS und Windows wird clang verwendet.

⟨RS3⟩ Der Algorithmus sollte auf Linux, MacOS und Windows funktionieren.

Die Algorithmen unterteilen sich für Linux und MacOS oder für Windows. Diese funktionieren auf dem jeweiligen Betriebssystem.

⟨RS4⟩ Es soll gezeigt werden, wie schnell die Operationen auf den Datenstrukturen ausgeführt werden.

Wir erstellen eine UI. Der Code für die UI ist in JavaFX geschrieben. In der UI kann man auswählen zwischen den Datenstrukturen, der Anzahl an Threads und der Anzahl an Operationen die durchgeführt werden sollen. Diese werden dann dem Benchmark übergeben, gemessen und der UI zurückgegeben.

⟨RS5⟩ Die Software soll visualisiert werden.

Die UI visualisiert die Daten aus dem Benchmark und gibt die Gesamtzahl an Operationen pro Sekunde in einem Liniendiagramm an. Das Diagramm enthält zum Vergleich die Werte von der lock-base Datenstruktur und der lock-free Datenstruktur. Zudem werden zwei zusätzliche Diagramme erstellt, diese zeigen die jeweilige Latenz mit Max- und Min-Wert an.

9.3 Kann-Kriterien

Kann-Kriterien: Leistungen die enthalten sein können, denen der Auftraggeber jedoch neutral gegenüber steht.

⟨RC1⟩ Ladebalken für die UI implementieren

Bei der Berechnung der Vergleichswerte für die Visualisierung kann ein Ladebalken implementiert. Aufgrund dessen wird auch bei größeren Operationsmengen mit längeren Wartezeiten sichergestellt, dass das Programm nicht abgestürzt ist und weiter lädt.

⟨RC2⟩ Speicherfunktion der Visualisierung als Bilddatei

Für die Visualisierung könnte ein Button mit Speicherfunktion implementiert werden. Dieser speichert eine Screenshot-ähnliche Datei der Diagramme.

⟨RC3⟩ Micheals List als lock-free implementieren und testen.

Bisher nicht implementiert, jedoch existieren bereits die nötigen Schnittstellen. Die Tests werden wie in RM4-6 ablaufen.

⟨RC4⟩ Micheals List als lock-base implementieren und testen.

Bisher nicht implementiert. Tests wie in RM4-6. Wenn die Datenstruktur als lock-base und lock-free implementiert wurde, kann Sie auch der UI hinzugefügt werden.

9.4 Abgrenzungskriterien

Abgrenzungskriterien: Leistungen die explizit nicht umgesetzt werden.

Hier ist zu verdeutlichen, welche Ziele mit dem Produkt bewusst nicht erreicht werden sollen oder können.

⟨RW1⟩ Es wird kein Handbuch mitgeliefert.

⟨RW2⟩ Die Funktionen der Library sollen nicht optimiert sein.

⟨RW3⟩ Auf spezifische Hardware wird nicht geachtet.

10 Glossar

Hier werden Fachbegriffe erklärt.

Begriff	Definition
Aktivitätsdiagramm	Graphische Darstellung von Software
Allozieren	Zuweisen von Speicherplatz
Atomare Operationen	Operation nur als Ganzes erfolgreich ist oder fehlschlägt
Betriebssystem	Zusammenarbeit von Software und Hardware
Compare and Swap (CAS)	Atomare Vergleichsoperation
Datenstruktur	Objekt, in dem Daten gespeichert werden
Deadlock	mehrere Prozesse blockieren sich gegenseitig
Hardwarekomponenten	Physische Komponenten eines Systems
Implementieren	Erzeugen eines funktionsfähigen Programms
JavaFX	Framework zur Erstellung von Java-Applikationen
Library	Sammlung ähnlicher Objekte
Linux	Ein freies Open Source-Betriebssystem
List	Datenstruktur
Lockfrei	Ohne Verwendung von Locking Algorithmen
Mac OS	Von Apple entwickeltes Betriebssystem
Mehrkernsysteme	System mit mehreren Prozessor-Kernen
Node	Element/Knoten
Queue	Datenstruktur
Sequenzdiagramm	Graphische Darstellung von Interaktionen
Set	Datenstruktur
Software	Computerprogramme
Thread	Aktivitätsträger in der Abarbeitung eines
Programms (Teil eines Prozessors)	
Thread-Sicherheit	Konsistenz auch bei Mehrkernbetrieb
Visualisierung	IdR Daten und Zusammenhänge graphisch darstellen
Wettlaufsituation	Ergebniss hängt von Prozessreihenfolge ab
Windows	Von Microsoft entwickeltes Betriebssystem